

Foundation Models and Generative Artificial Intelligence

ARIN 5203

Instructor: Dr. Qifeng Chen

Today:

Generative Models

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.



→ Cat

Classification

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.



A cat sitting on a suitcase on the floor

Image captioning

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.



DOG, DOG, CAT

Object Detection

Supervised vs Unsupervised Learning

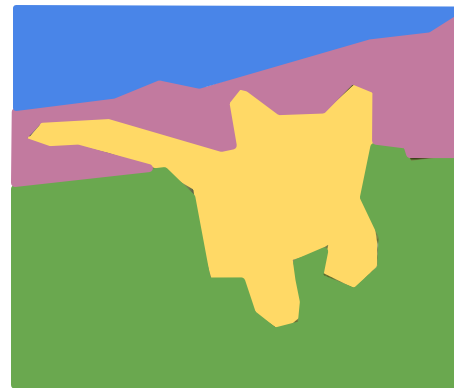
Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.



GRASS, CAT,
TREE, SKY

Semantic Segmentation

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

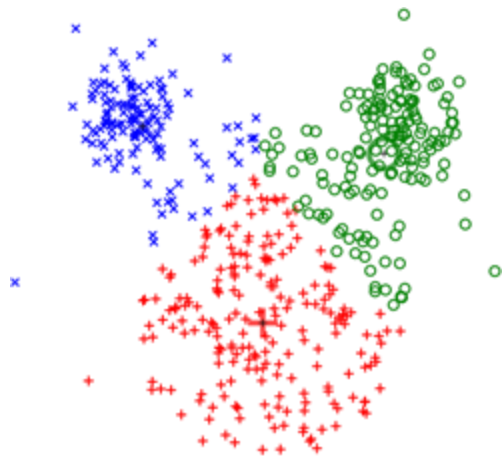
Data: x

Just data, no labels!

Goal: Learn hidden structure in data

Examples: Clustering, dimensionality reduction, density estimation, etc.

Supervised vs Unsupervised Learning



K-means clustering

Unsupervised Learning

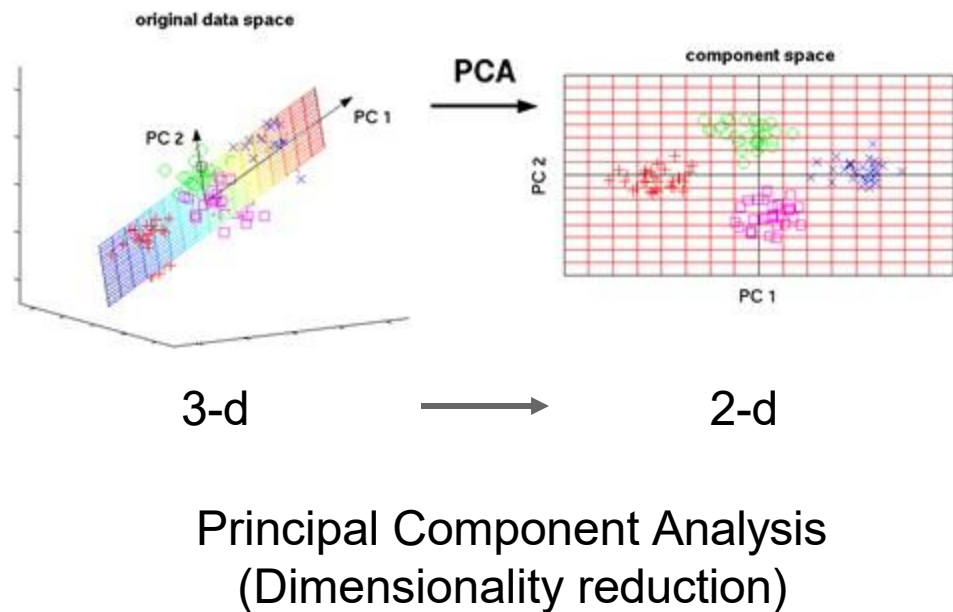
Data: x

Just data, no labels!

Goal: Learn hidden structure in data

Examples: Clustering, dimensionality reduction, density estimation, etc.

Supervised vs Unsupervised Learning



Unsupervised Learning

Data: X

Just data, no labels!

Goal: Learn hidden structure in data

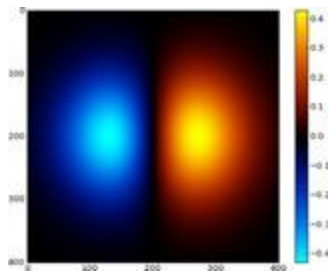
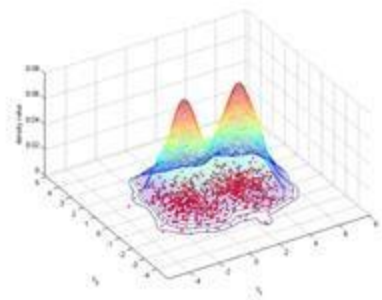
Examples: Clustering, dimensionality reduction, density estimation, etc.

Supervised vs Unsupervised Learning



Figure copyright Ian Goodfellow, 2016. Reproduced with permission.

1-d density estimation



2-d density estimation

Modeling $p(x)$

Unsupervised Learning

Data: X

Just data, no labels!

Goal: Learn hidden structure in data

Examples: Clustering, dimensionality reduction, density estimation, etc.

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Data: x



Label: y
Cat

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Data: x



Label: y
Cat

Probability Recap:

Density Function

$p(x)$ assigns a positive number to each possible x ; higher numbers mean x is more likely.

Density functions are **normalized**:

$$\int_X p(x) dx = 1$$

Different values of x **compete** for density

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Data: x

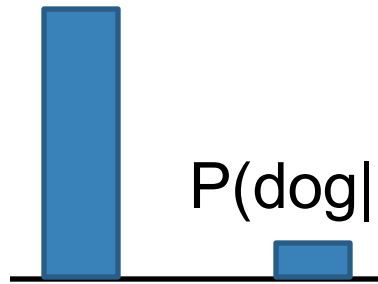


Label: y
Cat

$P(\text{cat} | \text{img})$



$P(\text{dog} | \text{img})$



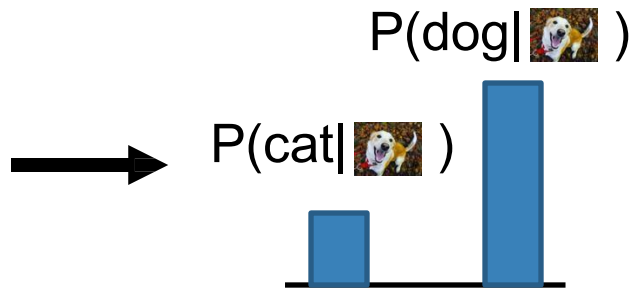
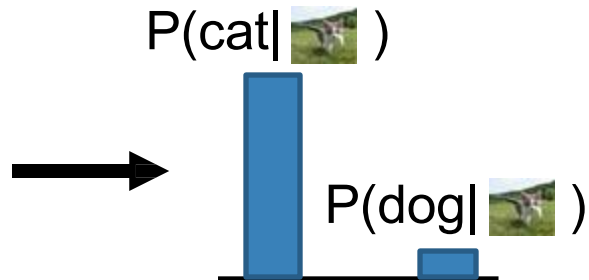
$$\int_X p(x) dx = 1$$

Different values of x
compete for density

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$



Possible **labels** for each image compete for probability.
No competition between **images**

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

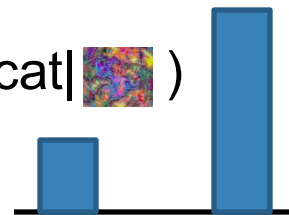


$P(\text{cat} | \text{img})$



$P(\text{dog} | \text{img})$

$P(\text{cat} | \text{img})$



No way to handle unreasonable inputs; must give a label distribution for all possible inputs

Generative vs Discriminative Models

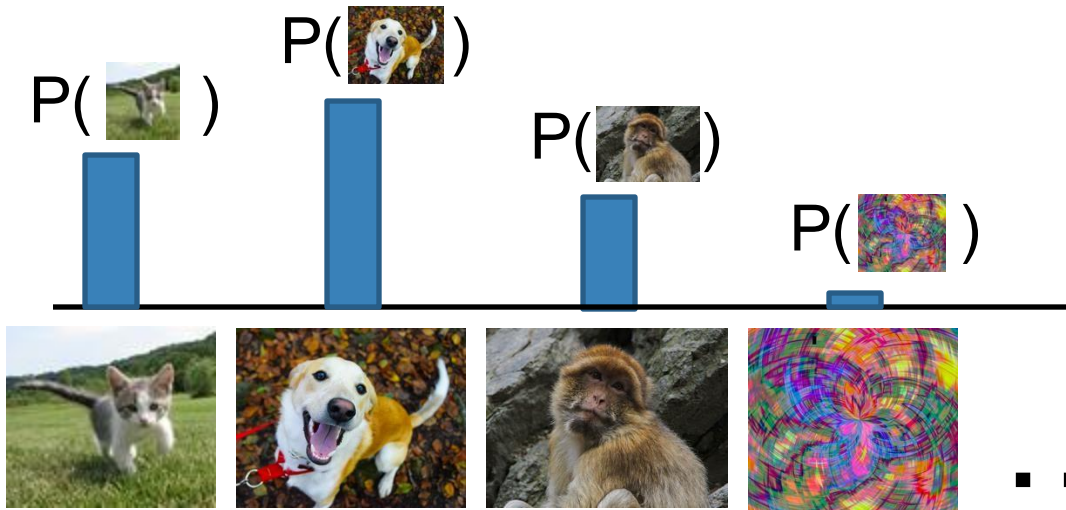
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$



All possible **images** compete for probability mass

Generative vs Discriminative Models

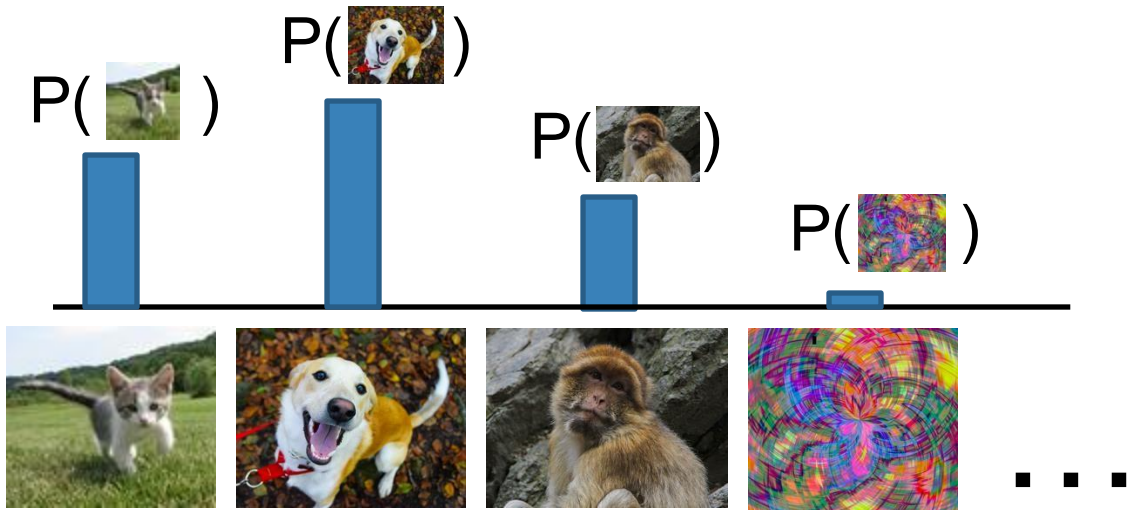
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$



All possible **images** compete for probability mass

Requires deep understanding: Is a dog more likely to sit or stand? Is a 3-legged dog more likely than a 3-armed monkey?

Generative vs Discriminative Models

Discriminative Model:

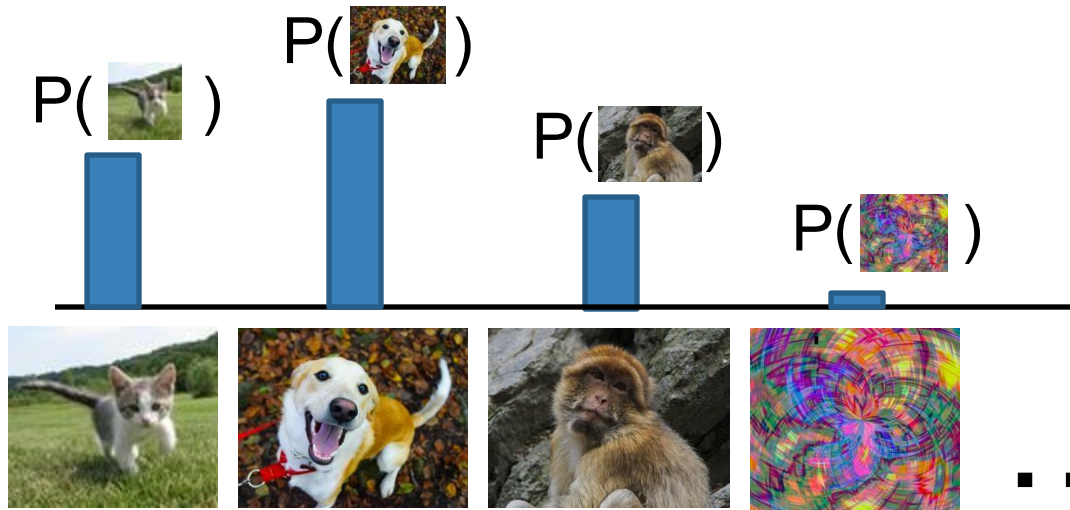
Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model:

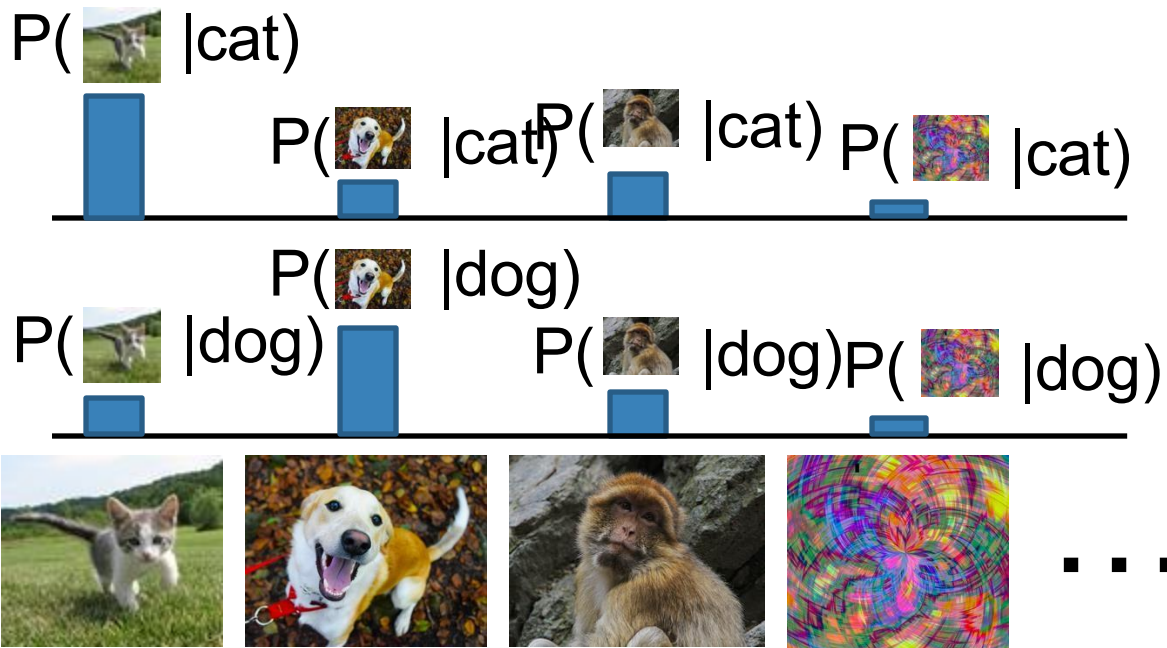
Learn $p(x|y)$



All possible **images** compete for probability mass

Model can “reject” unreasonable inputs by giving them small probability mass

Generative vs Discriminative Models



Conditional Generative Model: Learn $p(x|y)$

Each possible **label** induces a competition across all possible **images**

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Recall **Bayes' Rule**:

$$P(x | y) = \frac{P(y | x) P(x)}{P(y)}$$

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Recall **Bayes' Rule:**

$$\underbrace{P(x | y)}_{\substack{\text{Conditional} \\ \text{Generative Model}}} = \underbrace{\frac{P(y | x)}{P(y)}}_{\substack{\text{Prior over} \\ \text{labels}}} \underbrace{P(x)}_{\substack{\text{(Unconditional)} \\ \text{Generative Model}}}$$

Discriminative Model

We can build a conditional generative model from other components ... but not common in practice

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$



Assign labels to data
Feature learning (with labels)

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$



Assign labels to data
Feature learning (with labels)

Generative Model:

Learn a probability distribution $p(x)$



Detect outliers
Feature learning (without labels)
Sample to generate new data

Conditional Generative Model: Learn $p(x|y)$

Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$



Assign labels to data
Feature learning (with labels)

Generative Model:

Learn a probability distribution $p(x)$



Detect outliers
Feature learning (without labels)
Sample to generate new data

Conditional Generative Model: Learn $p(x|y)$



Assign labels while rejecting outliers
Sample to generate data from labels

Generative vs Discriminative Models

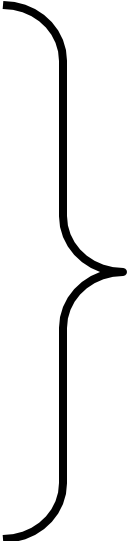
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$



"Generative models" means either of these; conditional generative models are most common in practice

Why Generative Models?

Modeling ambiguity: If there are many possible outputs x for an input y , we want to model $P(x | y)$

Language Modeling: Produce output text x from input text y

*Write me a short
rhyming poem about
generative models*



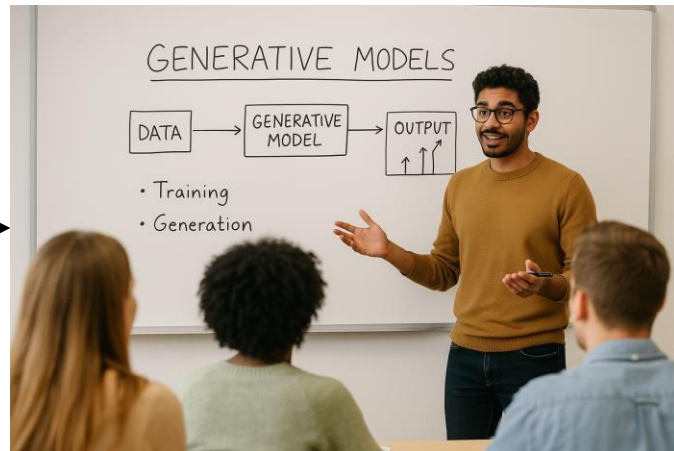
*They sample from a learned P ,
A distribution—structured, free.
Each token comes conditionally,
On all the ones that used to be.*

Why Generative Models?

Modeling ambiguity: If there are many possible outputs x for an input y , we want to model $P(x | y)$

Text to Image: Produce output image x from input text y

*Make me an image showing
a person teaching a class on
generative models in front of
a whiteboard*



Why Generative Models?

Modeling ambiguity: If there are many possible outputs x for an input y , we want to model $P(x | y)$

Image to Video: What happens next?



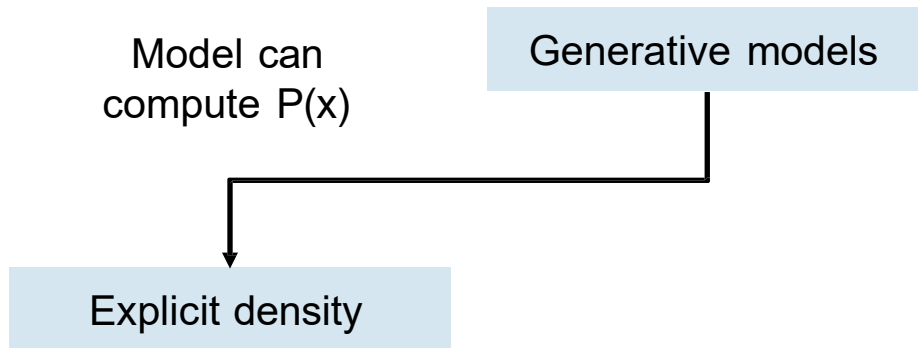
Taxonomy of Generative Models

Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017

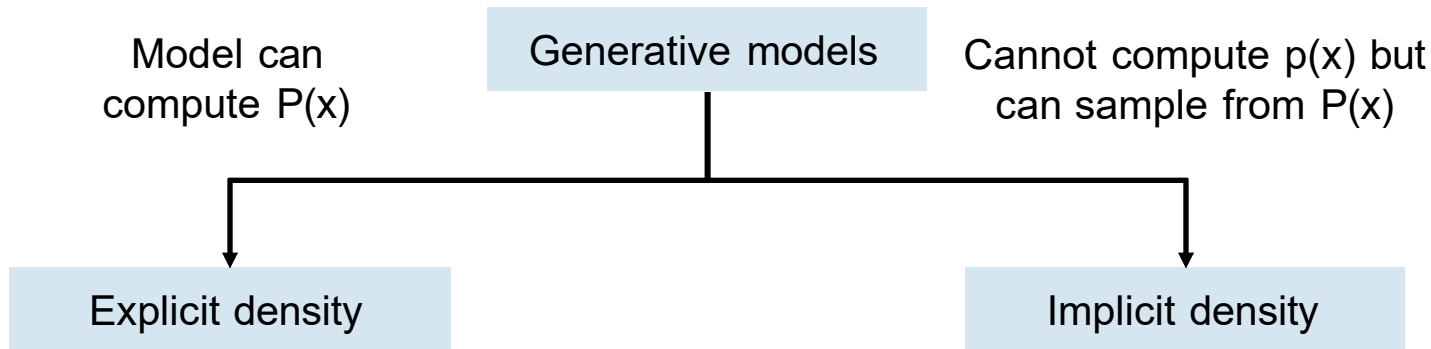
Generative models

Taxonomy of Generative Models

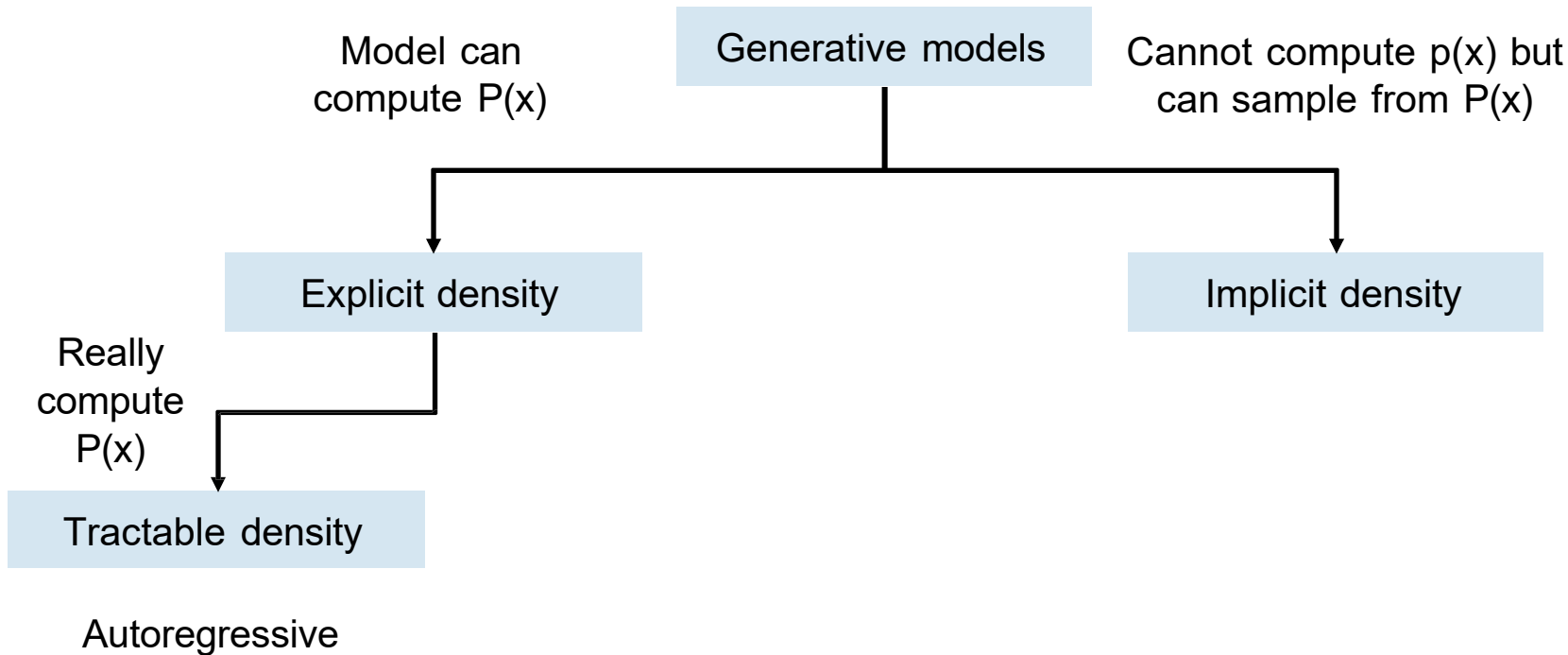
Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017



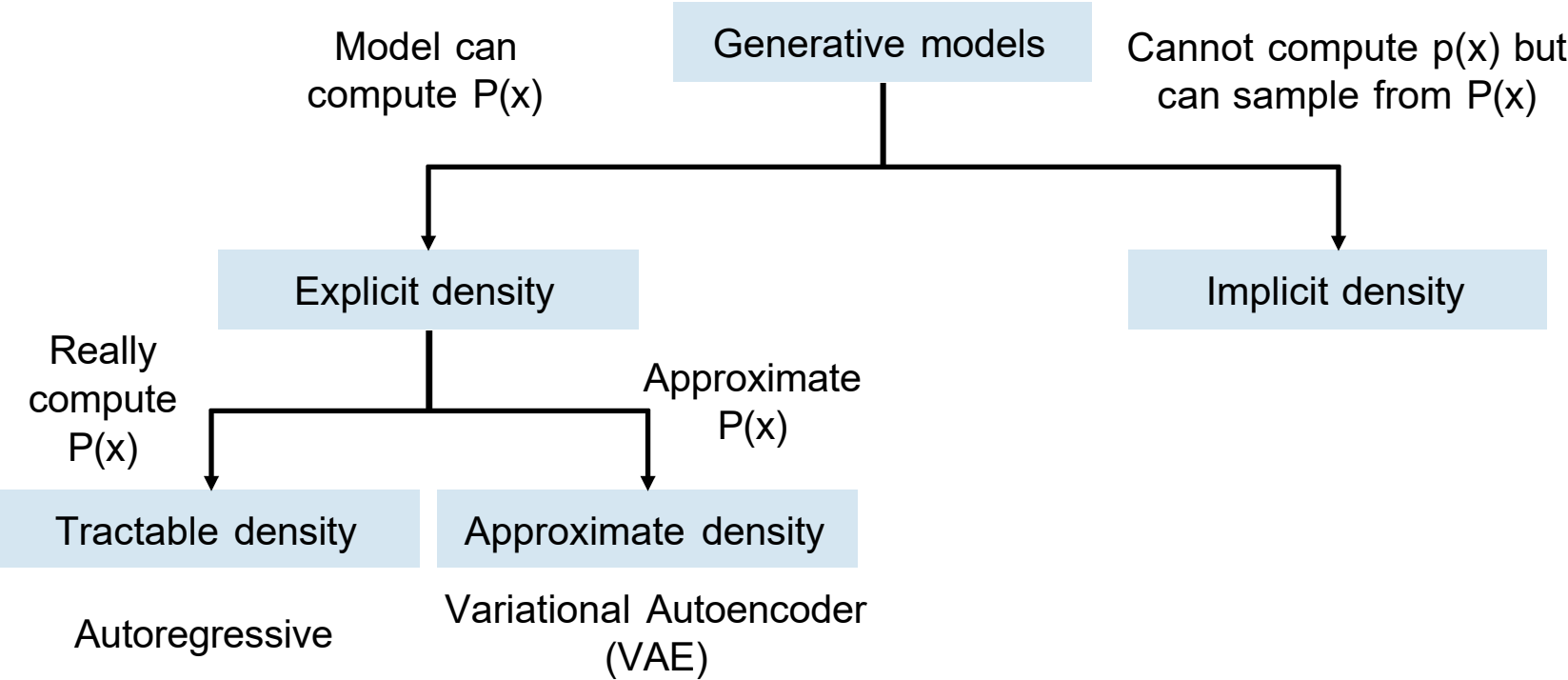
Taxonomy of Generative Models



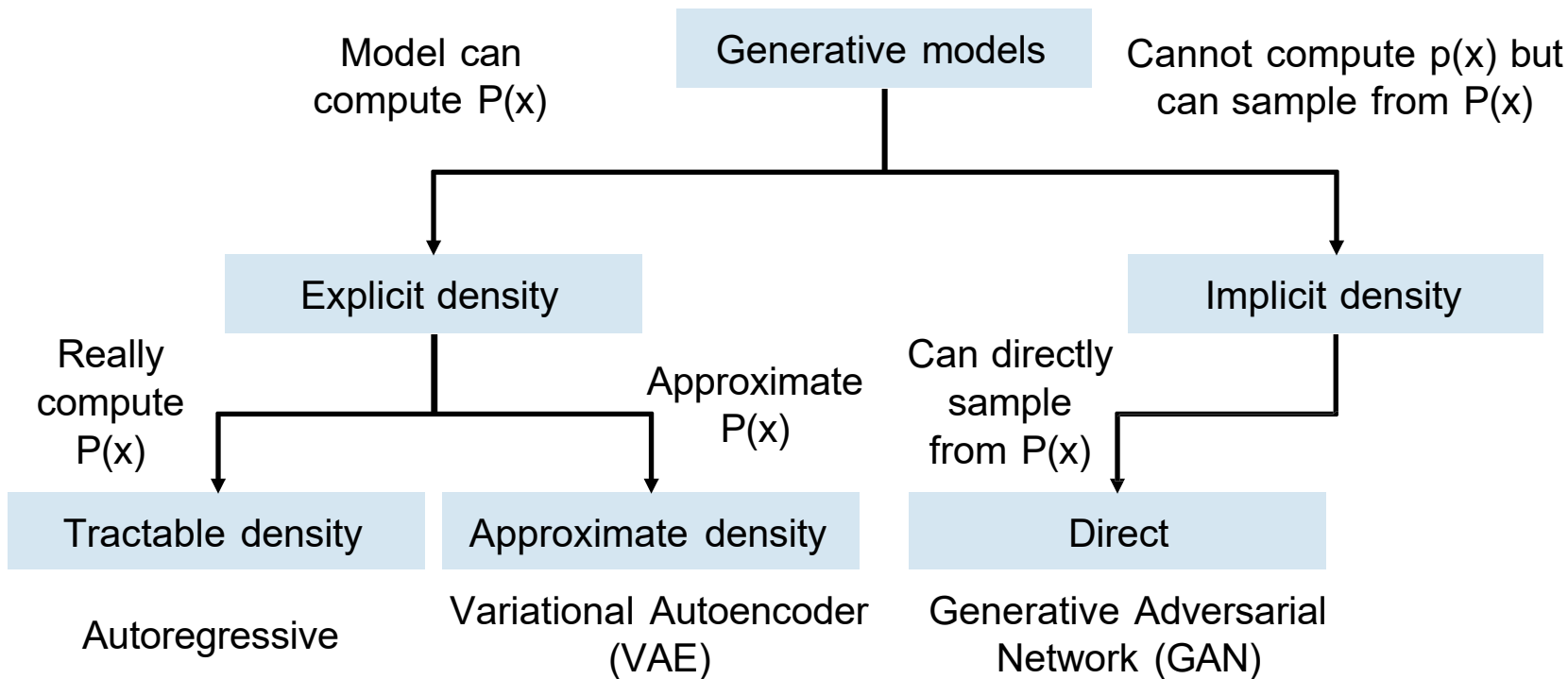
Taxonomy of Generative Models



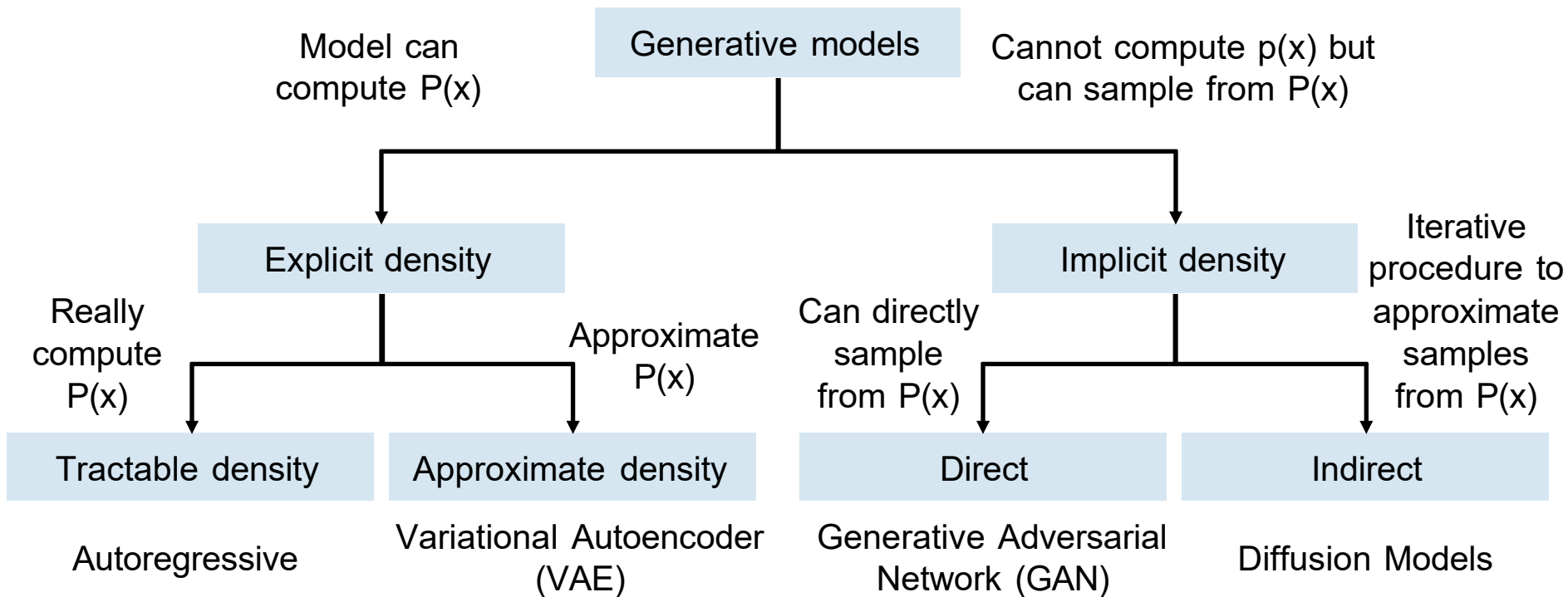
Taxonomy of Generative Models



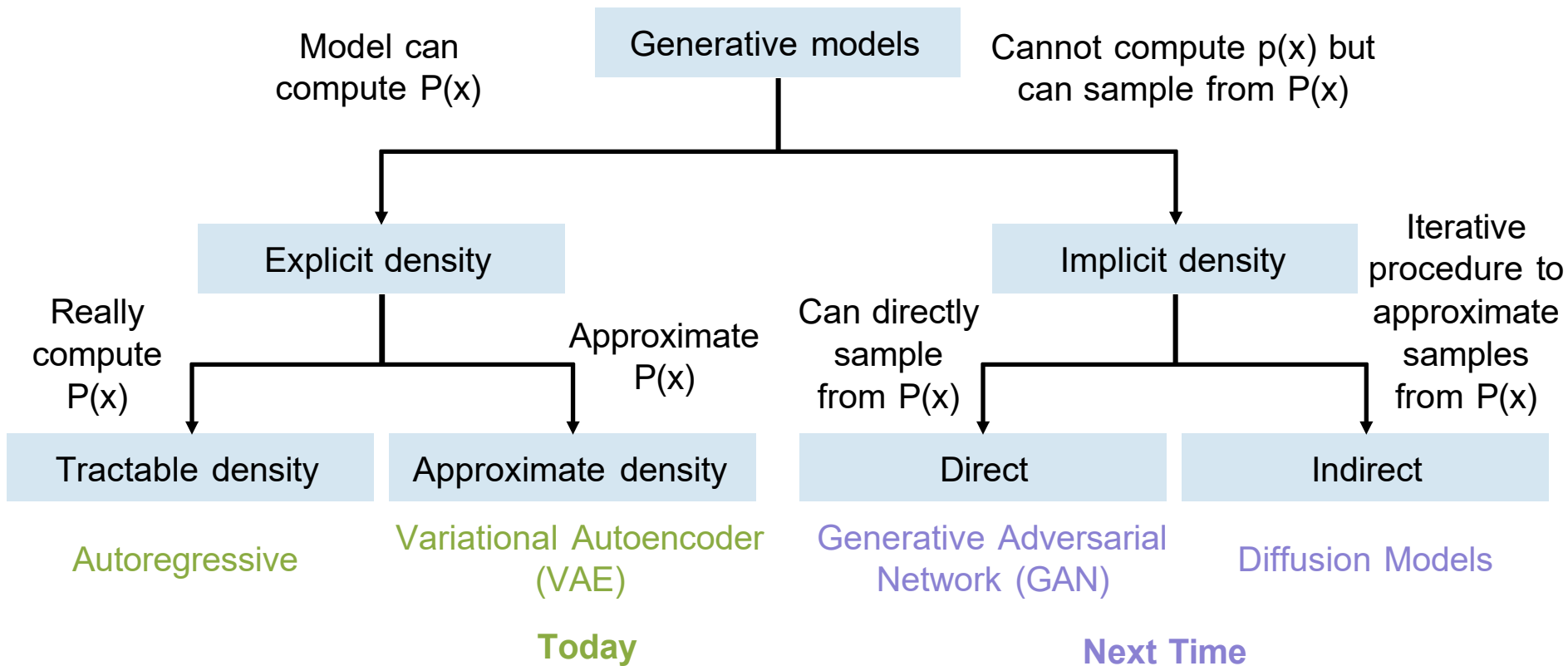
Taxonomy of Generative Models



Taxonomy of Generative Models



Taxonomy of Generative Models



Autoregressive Models

Maximum Likelihood Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Maximum Likelihood Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \dots, x^{(N)}$, train the model by solving:

$$W^* = \arg \max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

Maximum Likelihood Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \dots, x^{(N)}$, train the model by solving:

$$W^* = \arg \max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

$$= \arg \max_W \sum_i \log p(x^{(i)})$$

Log trick: Swap product and sum

Maximum Likelihood Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \dots x^{(N)}$, train the model by solving:

$$W^* = \arg \max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

$$= \arg \max_W \sum_i \log p(x^{(i)})$$

Log trick: Swap product and sum

$$= \arg \max_W \sum_i \log f(x^{(i)}, W)$$

This is our loss function.
maximize it with gradient descent

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$

Use the chain rule of probability:

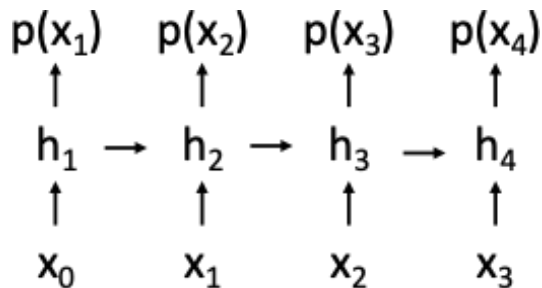
$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$

We have already seen this!



Use the chain rule of probability:

$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

Language modeling with RNN

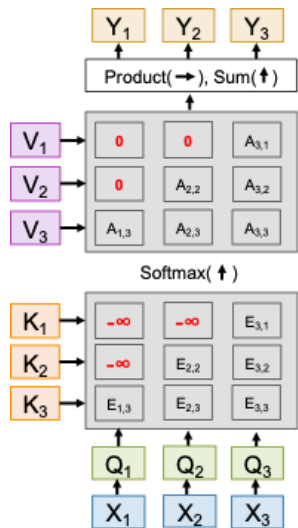
LLMs are Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence:

$$x = (x_1, x_2, \dots, x_T)$$

Language
modeling
with masked
Transformer



Use the chain rule of probability:

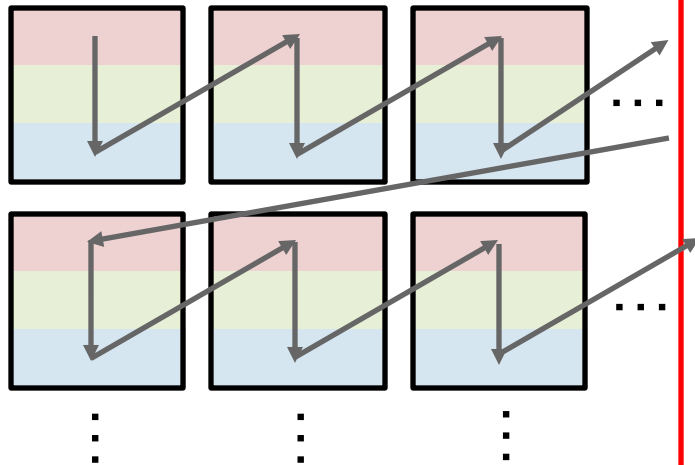
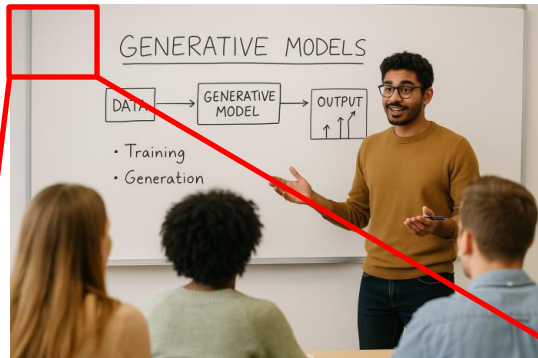
$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

Autoregressive Models of Images

Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values $[0 \dots 255]$

Model with an RNN or Transformer



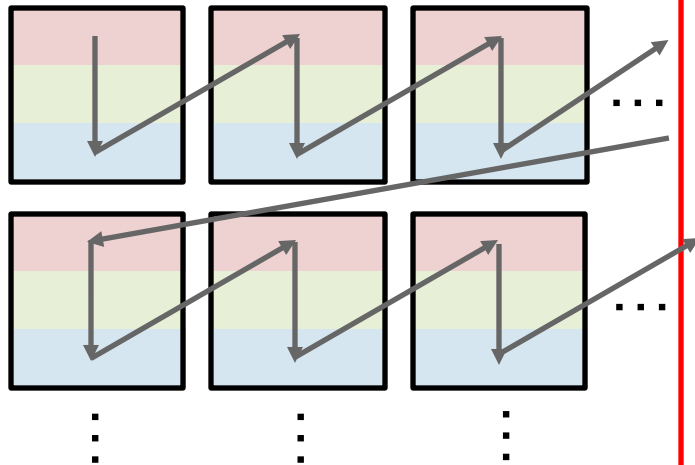
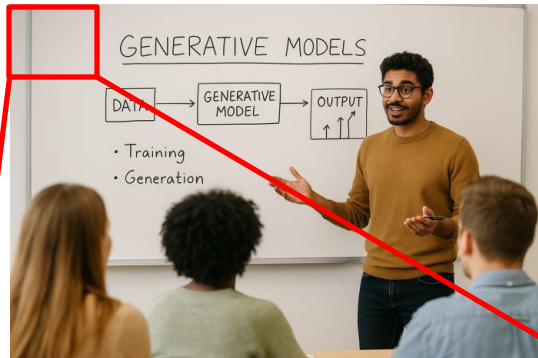
Autoregressive Models of Images

Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values $[0 \dots 255]$

Model with an RNN or Transformer

Problem: Too expensive. 1024×1024 image is a sequence of 3M subpixels



Autoregressive Models of Images

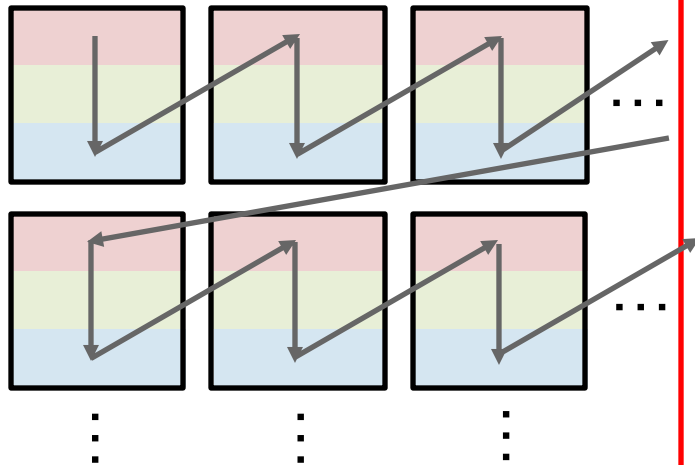
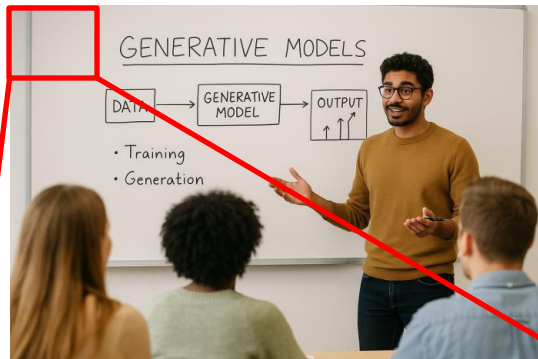
Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values $[0 \dots 255]$

Model with an RNN or Transformer

Problem: Too expensive. 1024×1024 image is a sequence of 3M subpixels

Solution (jumping ahead): Model as sequence of tiles, not sequence of subpixels



Variational Autoencoders (VAEs)

Variational Autoencoders

PixelRNN / PixelCNN explicitly parameterizes density function with a neural network, so we can train to maximize likelihood of training data:

$$p_W(x) = \prod_{t=1}^T p_W(x_t \mid x_1, \dots, x_{t-1})$$

Variational Autoencoders (VAE) define an **intractable density** that we cannot explicitly compute or optimize

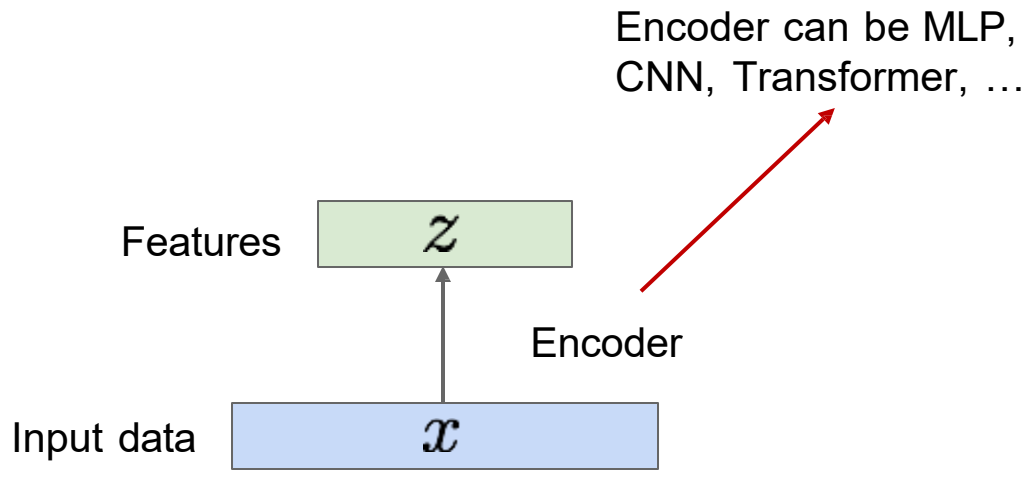
But we will be able to directly optimize a **lower bound** on the density

Variational Autoencoders (VAEs)

(Non-Variational) Autoencoders

Idea: Unsupervised method for learning to extract features z from inputs x , without labels

Features should extract useful information
(object identity, appearance, scene type, etc)
that can be used for downstream tasks

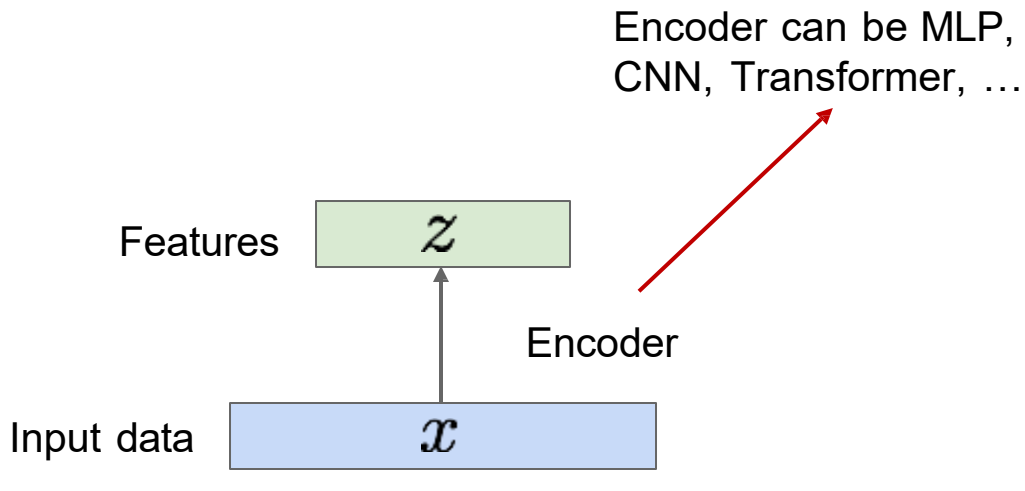


Input Data

(Non-Variational) Autoencoders

Problem: How can we learn without labels?

Features should extract useful information
(object identity, appearance, scene type, etc)
that can be used for downstream tasks



Input Data

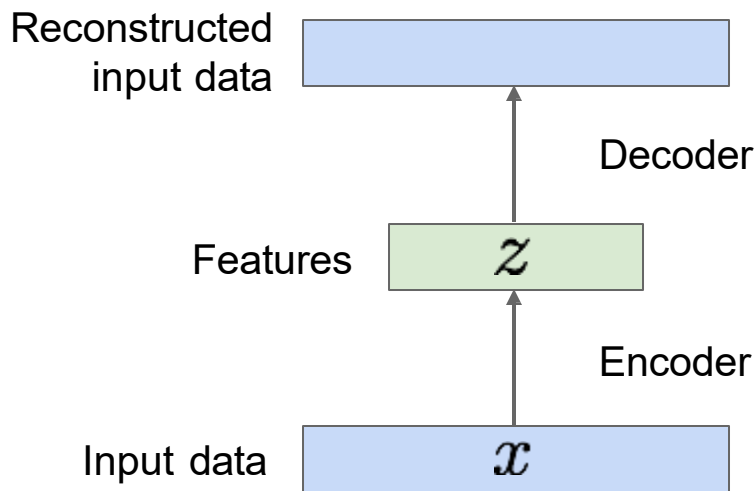
(Non-Variational) Autoencoders

“Autoencoding” =
Encoding yourself

Problem: How can we learn without labels?

Solution: Reconstruct the input data with a decoder.

Decoder can be MLP,
CNN, Transformer, ...

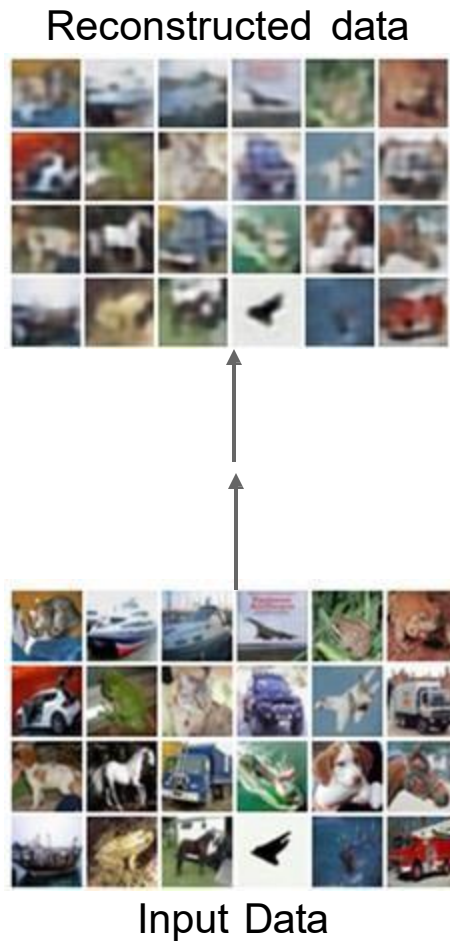
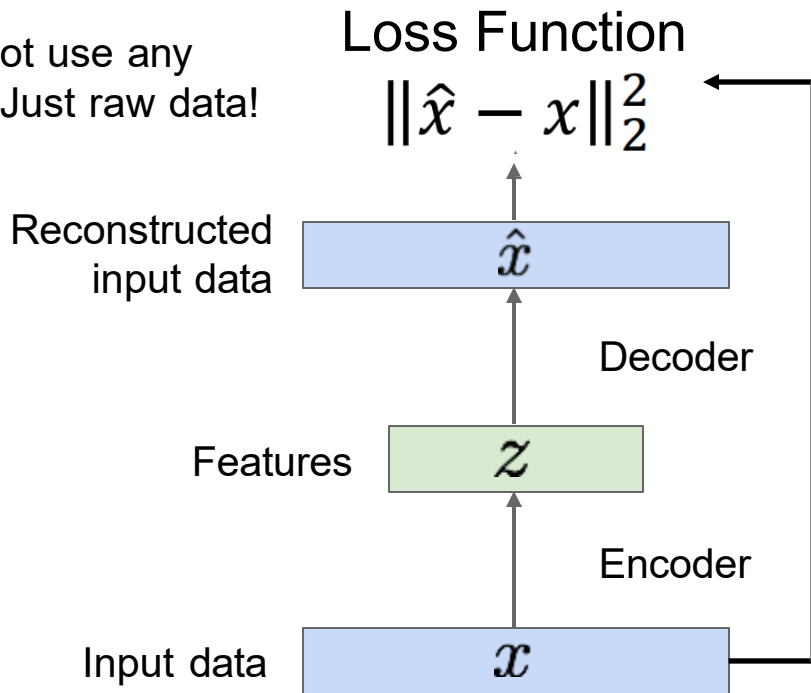


Input Data

(Non-Variational) Autoencoders

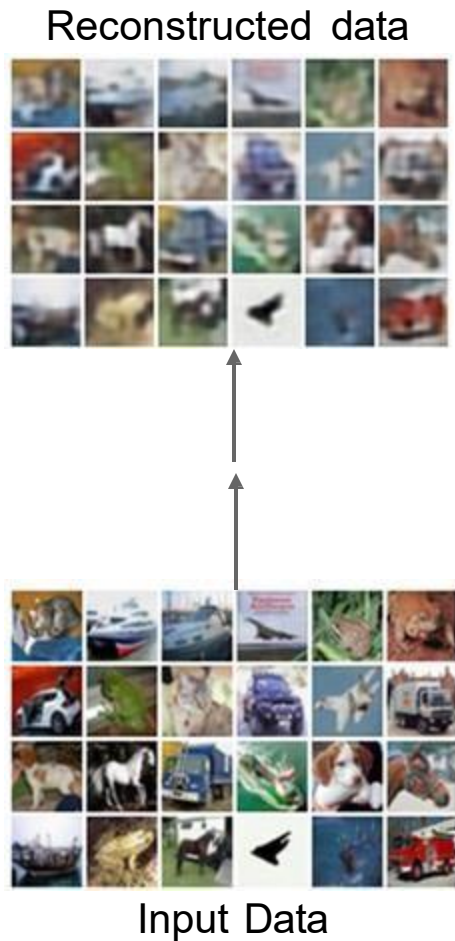
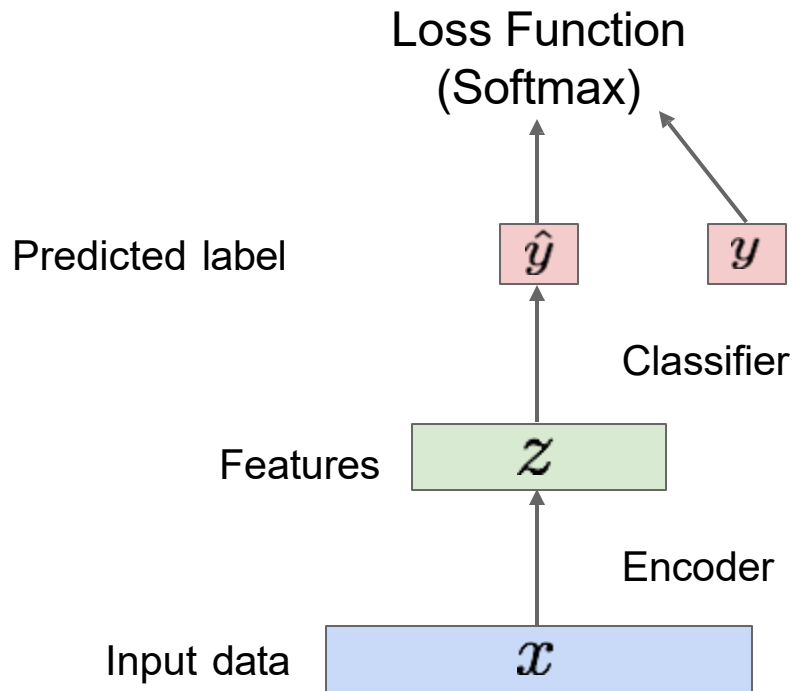
Loss: L2 distance between input and reconstructed data.

Does not use any
labels! Just raw data!



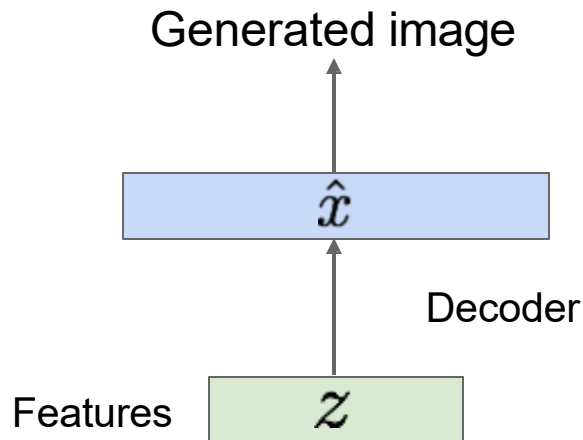
(Non-Variational) Autoencoders

After training, can use encoder for downstream tasks



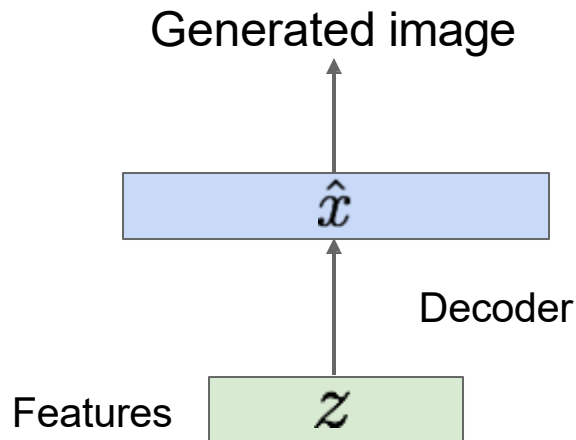
(Non-Variational) Autoencoders

If we could generate new z , could use the decoder to generate images



(Non-Variational) Autoencoders

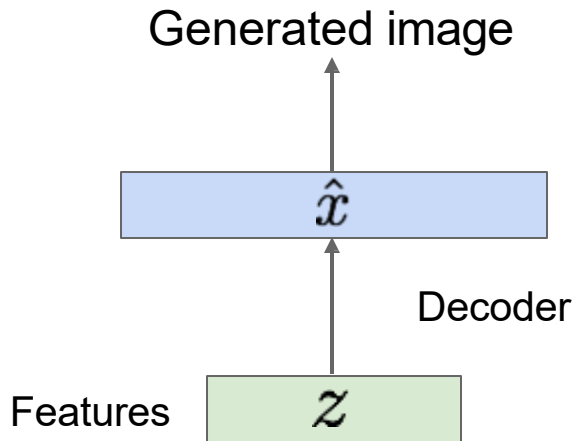
If we could generate new z , could use the decoder to generate images



Problem: Generating new z is not any easier than generating new x

(Non-Variational) Autoencoders

If we could generate new z , could use the decoder to generate images



Problem: Generating new z is not any easier than generating new x

Solution: What if we force all z to come from a known distribution?

Variational Autoencoders (VAEs)

Kingma and Welling, Auto-Encoding Variational Bayes, ICLR 2014

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features $\mathbf{z}$ from raw data
2. Sample from the model to generate new data

Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation $\mathbf{z}$

Intuition: $\mathbf{x}$ is an image, $\mathbf{z}$ is latent factors used to generate $\mathbf{x}$: attributes, orientation, etc.

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

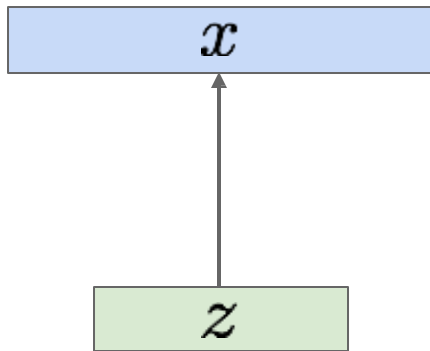
After training, sample new data like this:

Sample from
conditional

$$p_{\theta^*}(x \mid z^{(i)})$$

Sample z
from prior

$$p_{\theta^*}(z)$$



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation $\mathbf{z}$

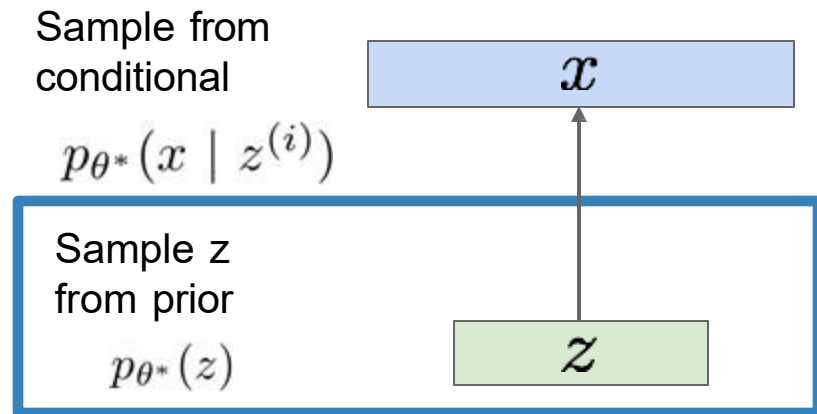
Intuition: $\mathbf{x}$ is an image, $\mathbf{z}$ is latent factors used to generate $\mathbf{x}$: attributes, orientation, etc.

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

Intuition: x is an image, z is latent factors used to generate x : attributes, orientation, etc.

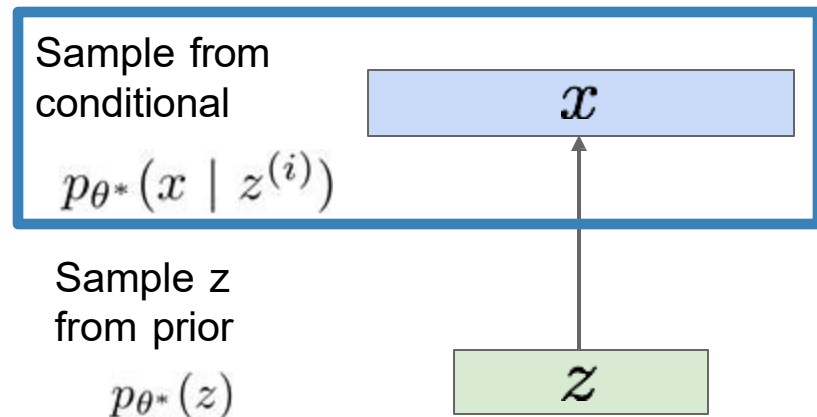
Assume simple prior $p(z)$, e.g. Gaussian

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

If we had a dataset of (x, z) then train a *conditional generative model* $p(x | z)$

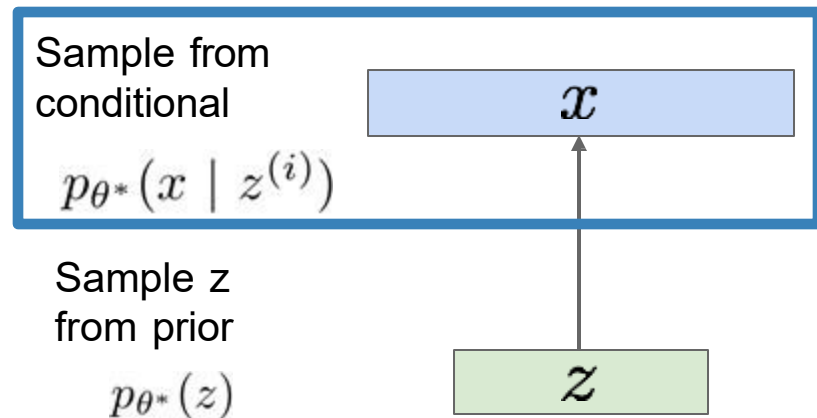
$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x|z)p_{\theta}(z) dz$$

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

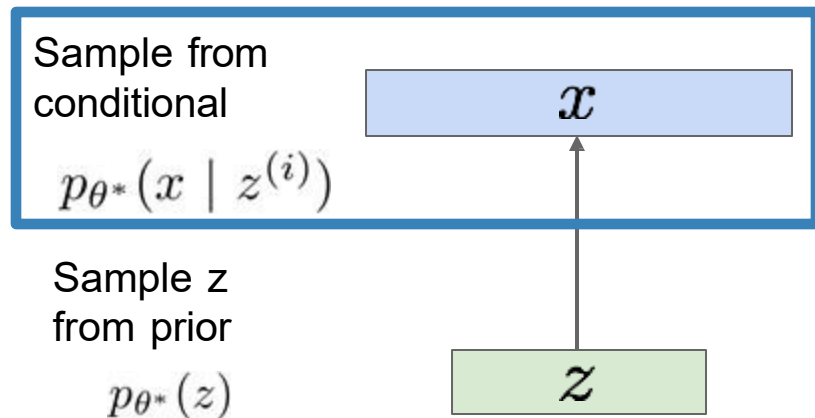
We don't observe z , so **marginalize**:

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

We don't observe z , so **marginalize**:

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x|z) p_{\theta}(z) dz$$

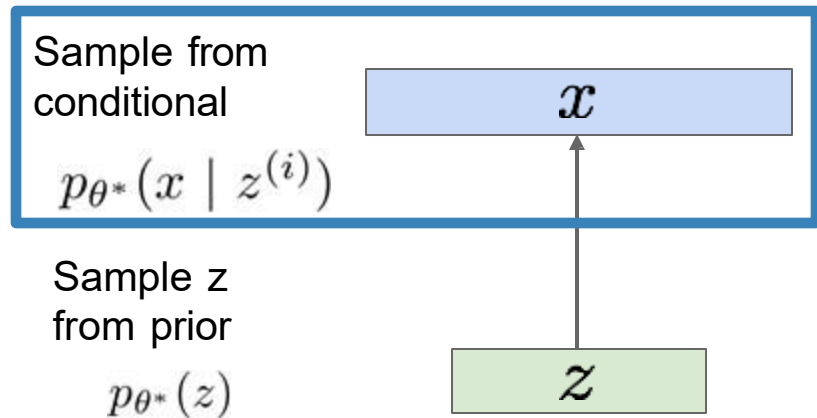
Ok, we can compute this with the decoder

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

We don't observe z , so **marginalize**:

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x|z) p_{\theta}(z) dz$$

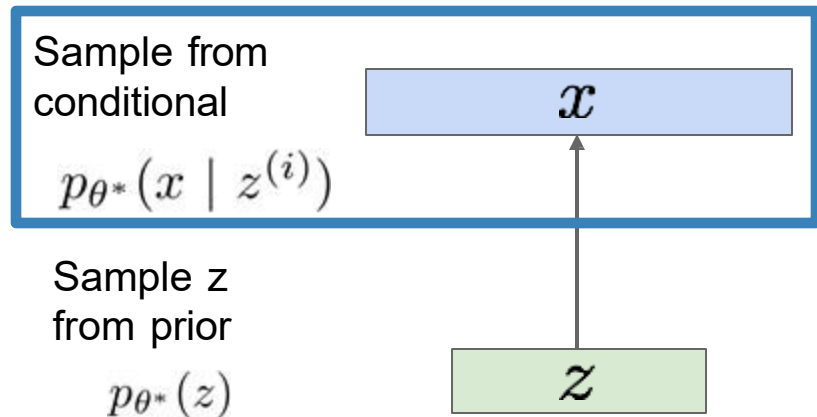
Ok, we assumed Gaussian prior for z

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

We don't observe z , so **marginalize**:

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x|z) p_{\theta}(z) d\mathbf{z}$$

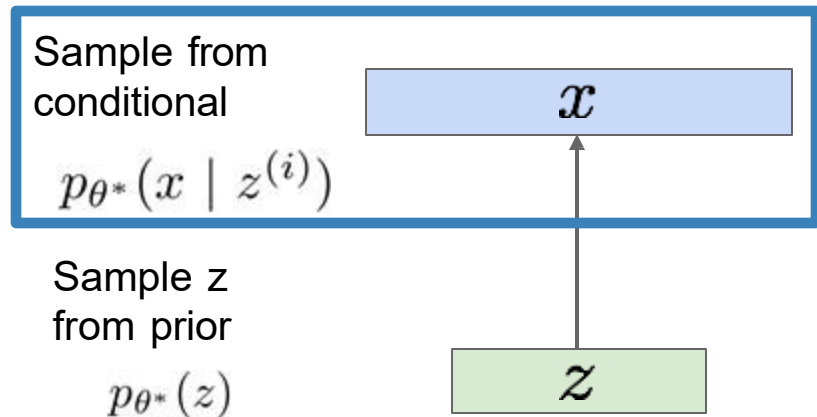
Problem, we can't integrate over all z

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation $\mathbf{z}$

How can we train this?

Basic idea: **maximum likelihood**

Another idea: Try Bayes' Rule:

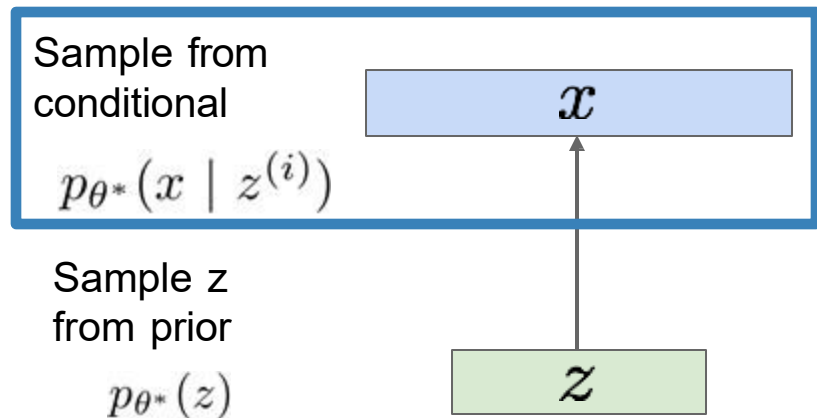
$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)}$$

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

Another idea: Try Bayes' Rule:

$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)}$$

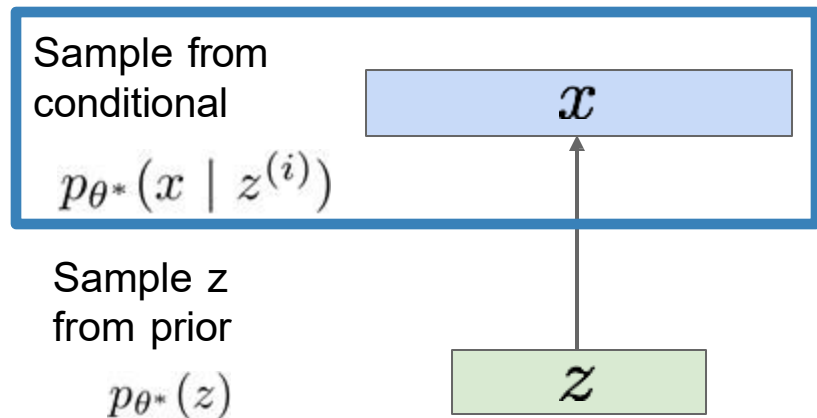
Ok, we can compute this with the decoder

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation $\mathbf{z}$

How can we train this?

Basic idea: **maximum likelihood**

Another idea: Try Bayes' Rule:

$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)}$$

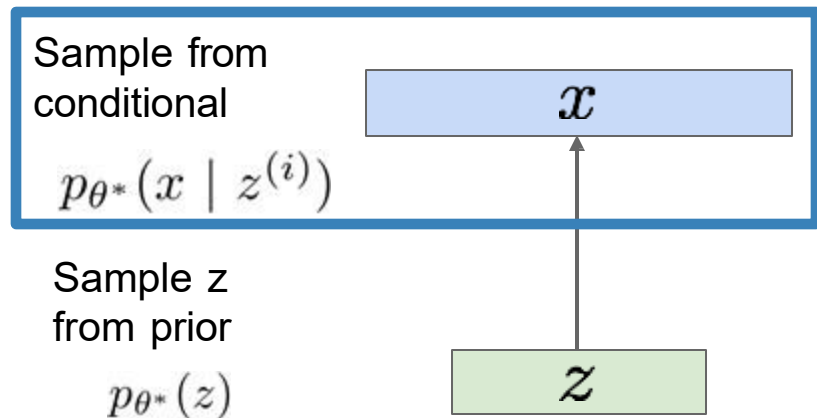
Ok, we assumed Gaussian prior for z

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

Another idea: Try Bayes' Rule:

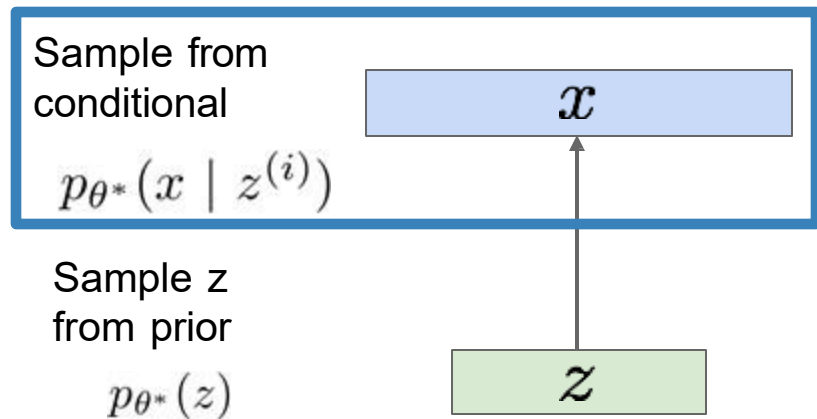
$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)} \quad \textbf{Problem: no way to compute this}$$

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation $\mathbf{z}$

How can we train this?

Basic idea: **maximum likelihood**

Another idea: Try Bayes' Rule:

$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)} \quad \text{Problem: no way to compute this}$$

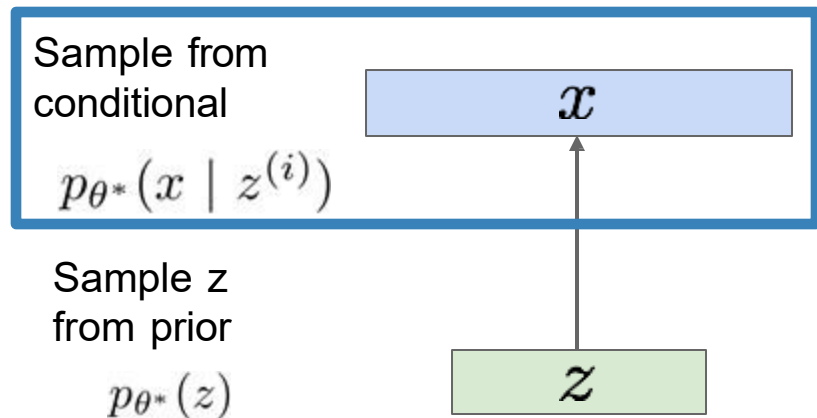
Solution: Train another network that learns $q_{\phi}(z | x) \approx p_{\theta}(z | x)$

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from unobserved (latent) representation z

How can we train this?

Basic idea: **maximum likelihood**

Another idea: Try Bayes' Rule:

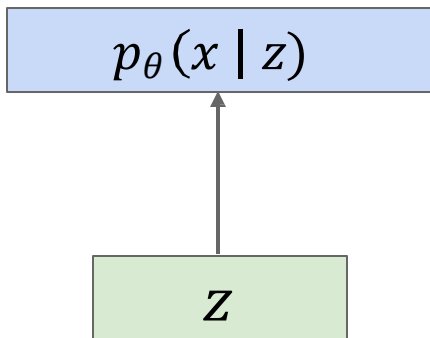
$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)} \approx \frac{p_{\theta}(x | z)p_{\theta}(z)}{q_{\phi}(z | x)}$$

Solution: Train another network that learns $q_{\phi}(z | x) \approx p_{\theta}(z | x)$

Variational Autoencoders

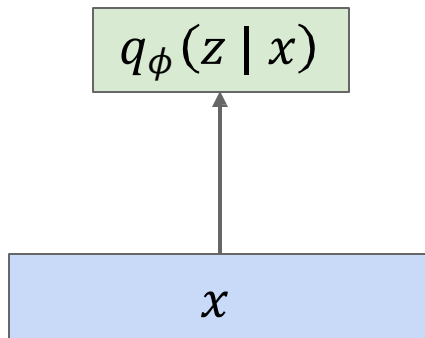
Decoder Network:

Input latent code z ,
Output distribution over data x



Encoder Network:

Input data x ,
Output distribution
over latent codes z



If we can ensure that
 $q_{\phi}(z | x) \approx p_{\theta}(z | x)$,

then we can approximate

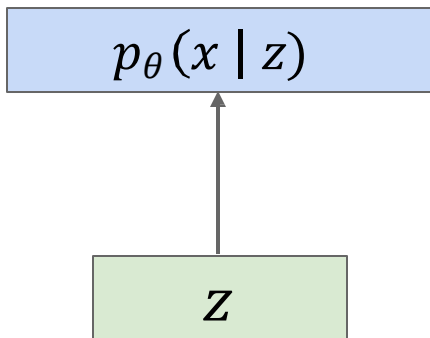
$$p_{\theta}(x) \approx \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)}$$

Idea: Jointly train both
encoder and decoder

Variational Autoencoders

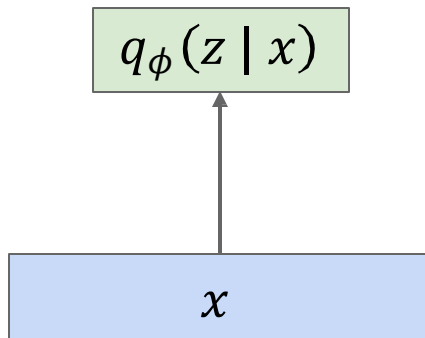
Decoder Network:

Input latent code z ,
Output distribution over data x



Encoder Network:

Input data x ,
Output distribution
over latent codes z



If we can ensure that
 $q_{\phi}(z | x) \approx p_{\theta}(z | x)$,

then we can approximate

$$p_{\theta}(x) \approx \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)}$$

Idea: Jointly train both
encoder and decoder

Aside: How to output probability
distributions from neural networks?

Network outputs mean (and std) of
a (diagonal) distribution

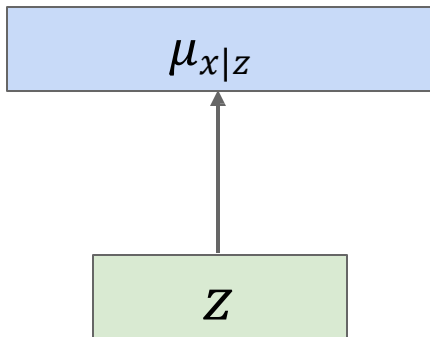
Variational Autoencoders

Decoder Network:

Input latent code z ,

Output distribution over data x

$$p_{\theta}(x | z) = N(\mu_{x|z}, \sigma^2)$$

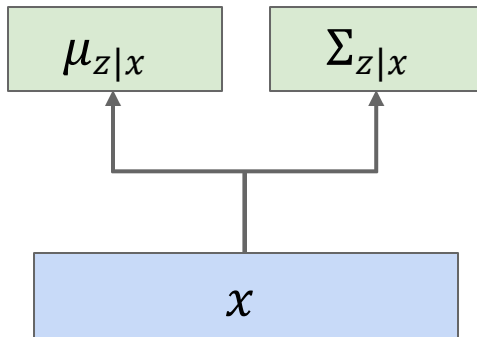


Encoder Network:

Input data x ,

Output distribution
over latent codes z

$$q_{\phi}(z | x) = N(\mu_{z|x}, \Sigma_{z|x})$$



If we can ensure that
 $q_{\phi}(z | x) \approx p_{\theta}(z | x)$,

then we can approximate

$$p_{\theta}(x) \approx \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)}$$

Idea: Jointly train both
encoder and decoder

Aside: How to output probability
distributions from neural networks?

Network outputs mean (and std) of
a (diagonal) distribution

Variational Autoencoders

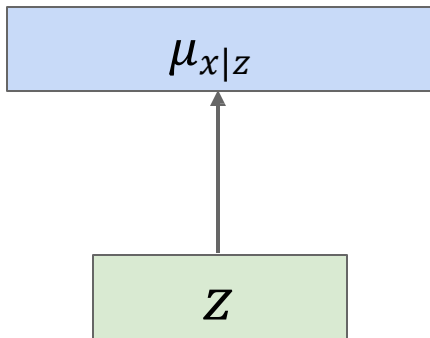
Decoder Network:

Input latent code z ,

Output distribution over data x

$$p_{\theta}(x | z) = N(\mu_{x|z}, \sigma^2)$$

$$\log p_{\theta}(x | z) = -\frac{1}{2\sigma^2} \|x - \mu\|_2^2 + C_2$$

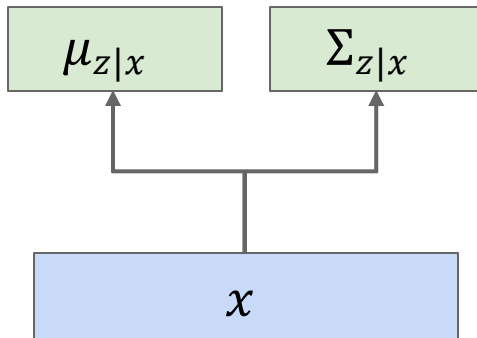


Encoder Network:

Input data x ,

Output distribution over latent codes z

$$q_{\phi}(z | x) = N(\mu_{z|x}, \Sigma_{z|x})$$



If we can ensure that $q_{\phi}(z | x) \approx p_{\theta}(z | x)$,

then we can approximate

$$p_{\theta}(x) \approx \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)}$$

Idea: Jointly train both encoder and decoder

Maximizing $\log p_{\theta}(x | z)$ is equivalent to minimizing L2 distance between x and network output!

Variational Autoencoders

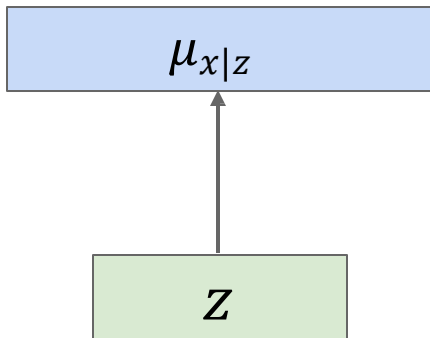
Decoder Network:

Input latent code z ,

Output distribution over data x

$$p_{\theta}(x | z) = N(\mu_{x|z}, \sigma^2)$$

$$\log p_{\theta}(x | z) = -\frac{1}{2\sigma^2} \|x - \mu\|_2^2 + C_2$$

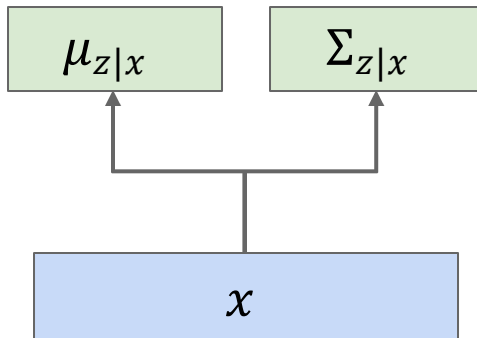


Encoder Network:

Input data x ,

Output distribution
over latent codes z

$$q_{\phi}(z | x) = N(\mu_{z|x}, \Sigma_{z|x})$$



If we can ensure that
 $q_{\phi}(z | x) \approx p_{\theta}(z | x)$,

then we can approximate

$$p_{\theta}(x) \approx \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)}$$

Idea: Jointly train both
encoder and decoder

Q: What's our
training objective?

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)}$$

Bayes' Rule

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

Multiply top and bottom by $q_{\phi}(z|x)$

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p(z)} + \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)}$$

Logarithms + rearranging

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p(z)} + \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)}$$

$$\log p_{\theta}(x) = E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x)]$$

We can wrap in an expectation since it doesn't depend on z

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$\log p_{\theta}(x) = E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x)]$$

We can wrap in an expectation since it doesn't depend on z

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right) + D_{KL}(q_{\phi}(z|x), p_{\theta}(z|x))$$

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z)) + D_{KL}(q_{\phi}(z|x), p_{\theta}(z|x))$$

Data reconstruction: $x \Rightarrow$ encoder $\Rightarrow$ decoder should reconstruct x
Can compute in closed form for Gaussians.

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right) + D_{KL}(q_{\phi}(z|x), p_{\theta}(z|x))$$

Prior: Encoder output should match prior over z .

Can compute in closed form for Gaussians.

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right) + D_{KL}(q_{\phi}(z|x), p_{\theta}(z|x))$$

Posterior Approximation: Encoder output $q_{\phi}(z|x)$ should match $p_{\theta}(z|x)$

We cannot compute this for Gaussians...

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right) + D_{KL}(q_{\phi}(z|x), p_{\theta}(z|x))$$

Posterior Approximation: Decoder output $q_{\phi}(z|x)$ should match $p_{\theta}(z|x)$

KL is ≥ 0 , so we can drop it to get a lower bound on likelihood

Variational Autoencoders (ELBO)

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z[\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right) + D_{KL}(q_{\phi}(z|x), p_{\theta}(z|x))$$

$\log p_{\theta}(x) \geq E_{z \sim q_{\phi}(z x)}[\log p_{\theta}(x z)] - D_{KL} \left(q_{\phi}(z x), p(z) \right)$	This is our VAE training objective
--	------------------------------------

Variational Autoencoders

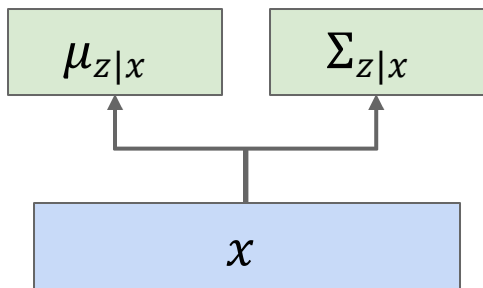
Jointly train **encoder** q and **decoder** p to maximize the **variational lower bound** on the data likelihood

Also called **Evidence Lower Bound (ELBo)**

$$\log p_{\theta}(x) \geq E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right)$$

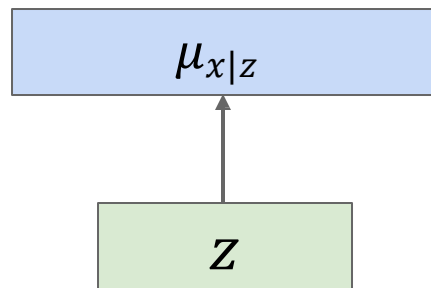
Encoder Network

$$q_{\phi}(z | x) = N(\mu_{z|x}, \Sigma_{z|x})$$



Decoder Network

$$p_{\theta}(x | z) = N(\mu_{x|z}, \sigma^2)$$



Variational Autoencoders: Training

Train by maximizing the
variational lower bound

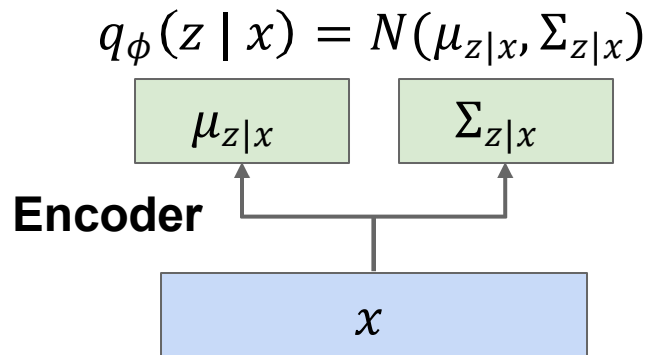
$$E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

1. Run input data through **encoder** to get distribution over z

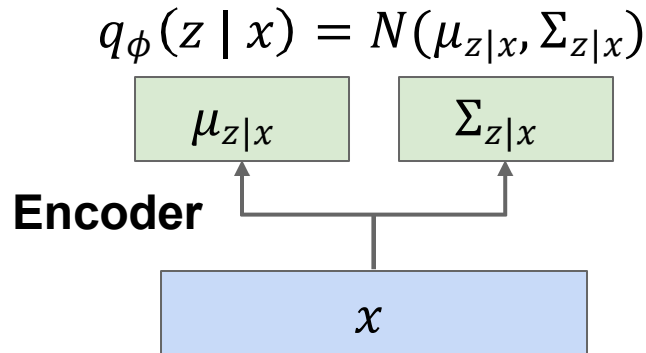


Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

1. Run input data through **encoder** to get distribution over z
2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)

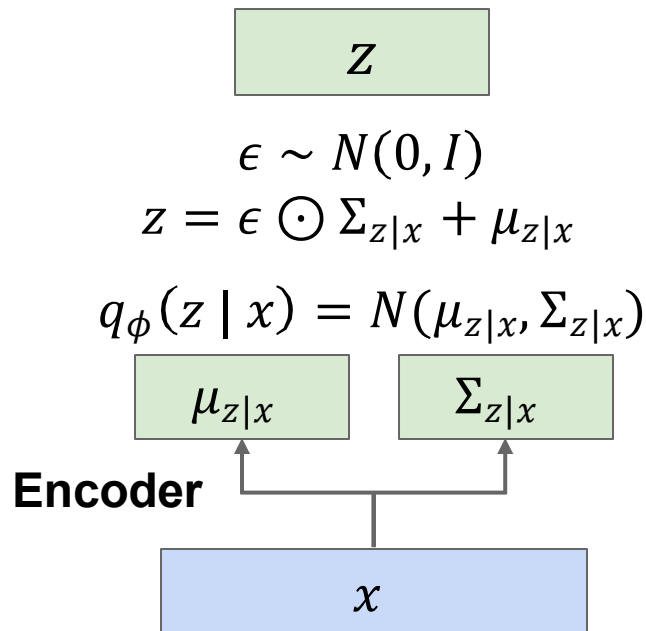


Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

1. Run input data through **encoder** to get distribution over z
2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)
3. Sample z from encoder output $q_{\phi}(z|x)$ (Reparameterization trick)

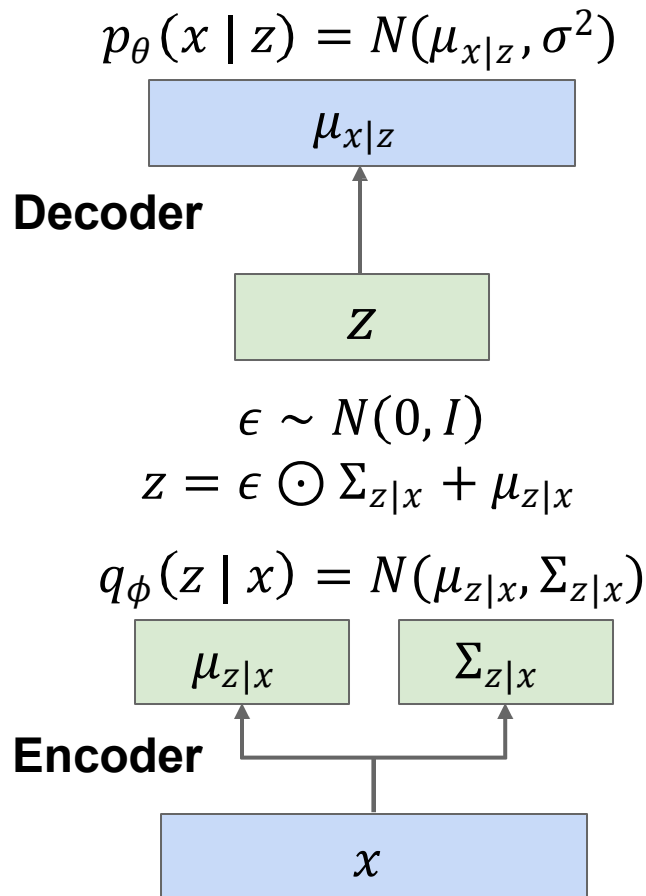


Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$E_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x), p(z))$$

1. Run input data through **encoder** to get distribution over z
2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)
3. Sample z from encoder output $q_\phi(z|x)$ (Reparameterization trick)
4. Run z through **decoder** to get predicted data mean

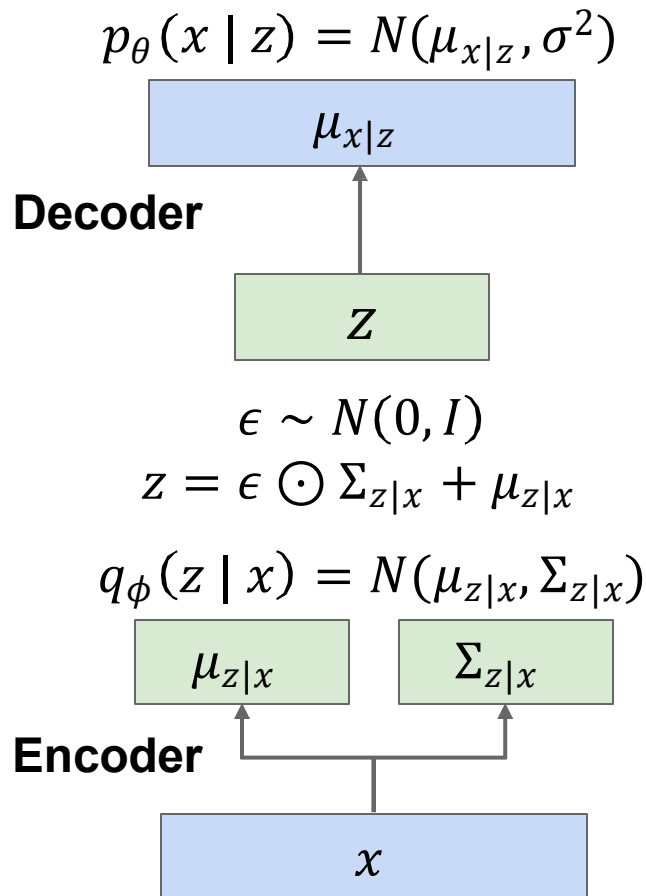


Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

1. Run input data through **encoder** to get distribution over z
2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)
3. Sample z from encoder output $q_{\phi}(z|x)$ (Reparameterization trick)
4. Run z through **decoder** to get predicted data mean
5. Reconstruction loss: predicted mean should match x in L2



Variational Autoencoders: Training

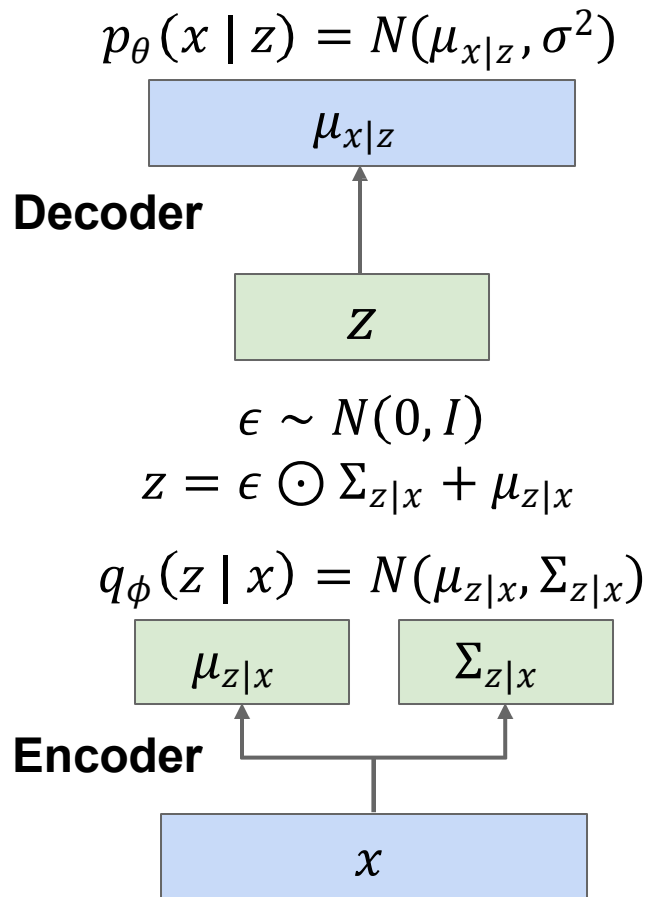
Train by maximizing the
variational lower bound

$$E_{z \sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x), p(z))$$

The loss terms fight against each other!

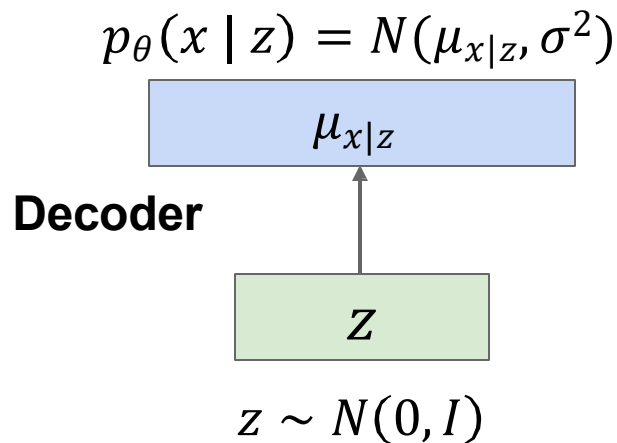
Reconstruction loss wants $\Sigma_{z|x} = 0$ and $\mu_{z|x}$ to be unique for each x , so decoder can deterministically reconstruct x

Prior loss wants $\Sigma_{z|x} = \mathbf{I}$ and $\mu_{z|x} = 0$ so encoder output is always a unit Gaussian



Variational Autoencoders: Sampling

1. Sample z from the prior
2. Run through decoder to get an image

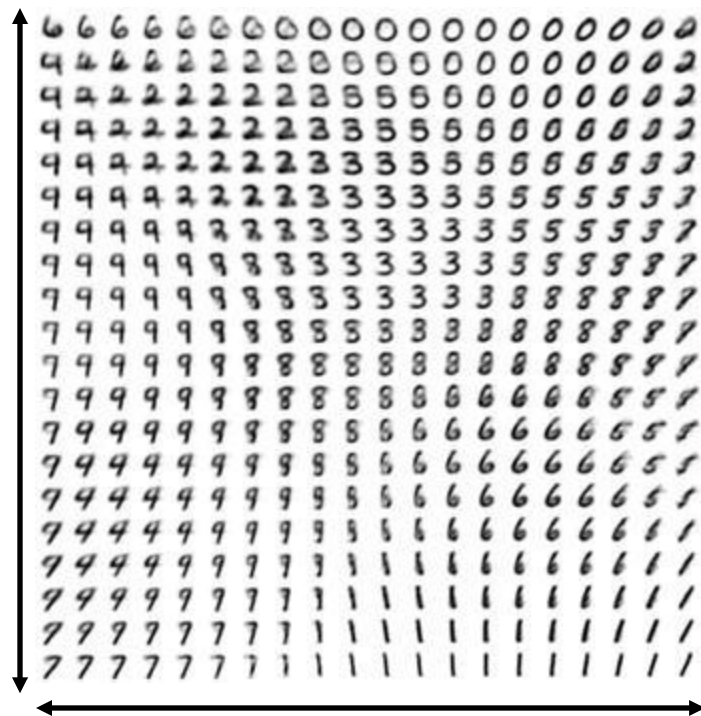


Variational Autoencoders: Disentangling

The diagonal prior on $p(z)$ causes dimensions of z to be independent

“Disentangling factors of variation”

Vary z_1



Recap: Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Data: x

Just data, no labels!

Goal: Learn hidden structure in data

Examples: Clustering, dimensionality reduction, density estimation, etc.

Recap: Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Data: x



Label: y
Cat

Density Function

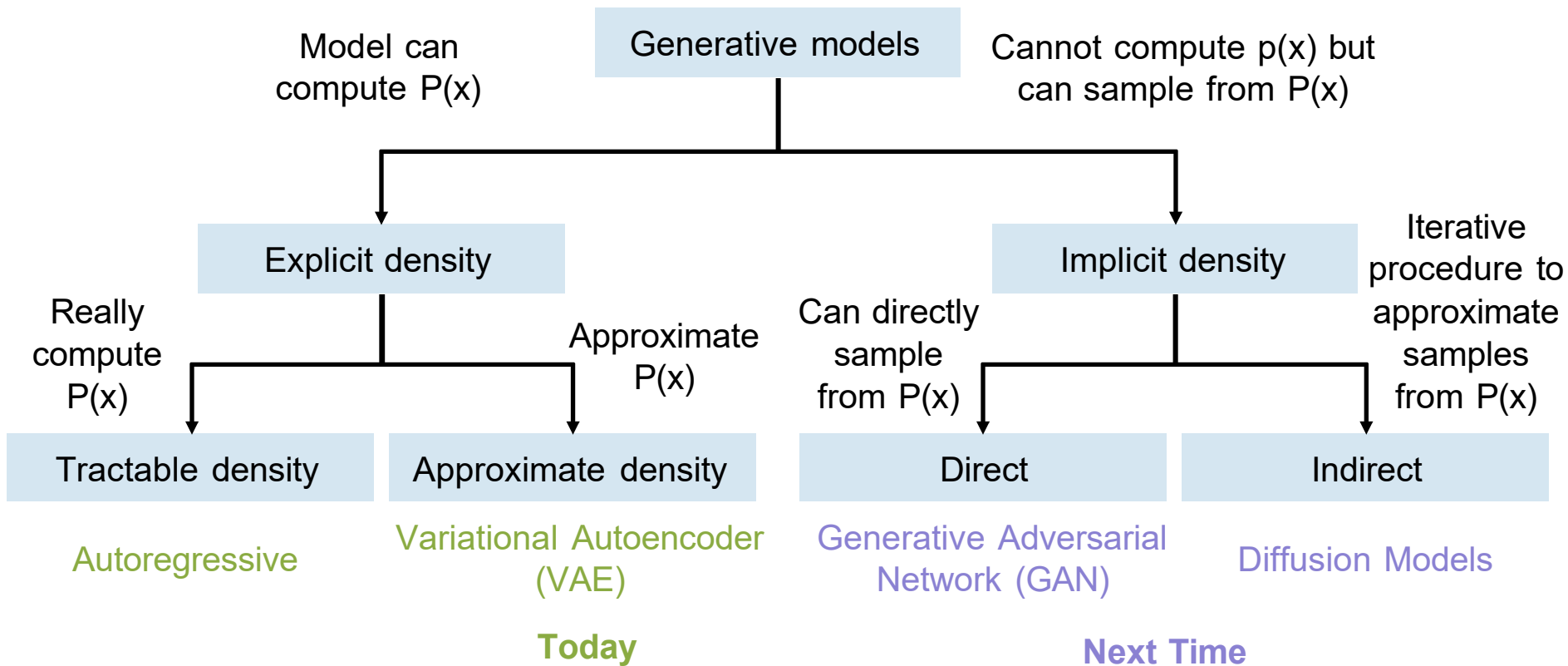
$p(x)$ assigns a positive number to each possible x ; higher numbers mean x is more likely.

Density functions are **normalized**:

$$\int_x p(x) dx = 1$$

Different values of x **compete** for density

Recap: Generative Models



Generative Adversarial Networks (GANs)

Generative Models So Far

Autoregressive Models directly maximize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

Variational Autoencoders introduce a latent z , and maximize a lower bound:

$$p_{\theta}(x) = \int_Z p_{\theta}(x|z)p(z)dz \geq E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

Generative Models So Far

Autoregressive Models directly maximize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

Variational Autoencoders introduce a latent z , and maximize a lower bound:

$$p_{\theta}(x) = \int_Z p_{\theta}(x|z)p(z)dz \geq E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

Generative Adversarial Networks give up on modeling $p(x)$, but allow us to draw samples from $p(x)$

Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{\text{data}}(x)$. Want to sample from p_{data}

Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{\text{data}}(x)$. Want to sample from p_{data}

Idea: Introduce a latent variable z with simple prior $p(z)$ (e.g. unit Gaussian)

Sample $z \sim p(z)$ and pass to a **Generator Network** $x = G(z)$

Then x is a sample from the **Generator distribution** p_G . Want $p_G = p_{\text{data}}$!

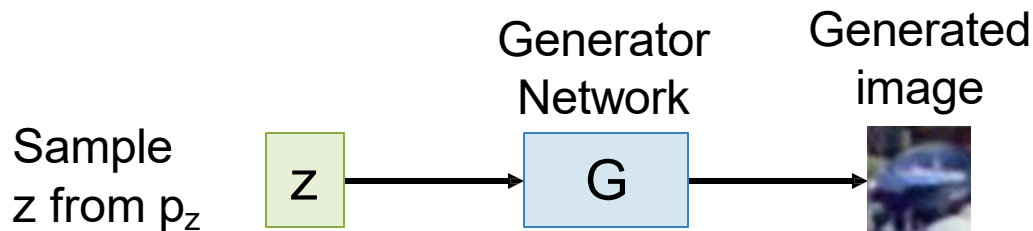
Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{\text{data}}(x)$. Want to sample from p_{data}

Idea: Introduce a latent variable z with simple prior $p(z)$ (e.g. unit Gaussian)

Sample $z \sim p(z)$ and pass to a **Generator Network** $x = G(z)$

Then x is a sample from the **Generator distribution** p_G . Want $p_G = p_{\text{data}}$!



Train **Generator Network** G to convert z into fake data x sampled from p_G

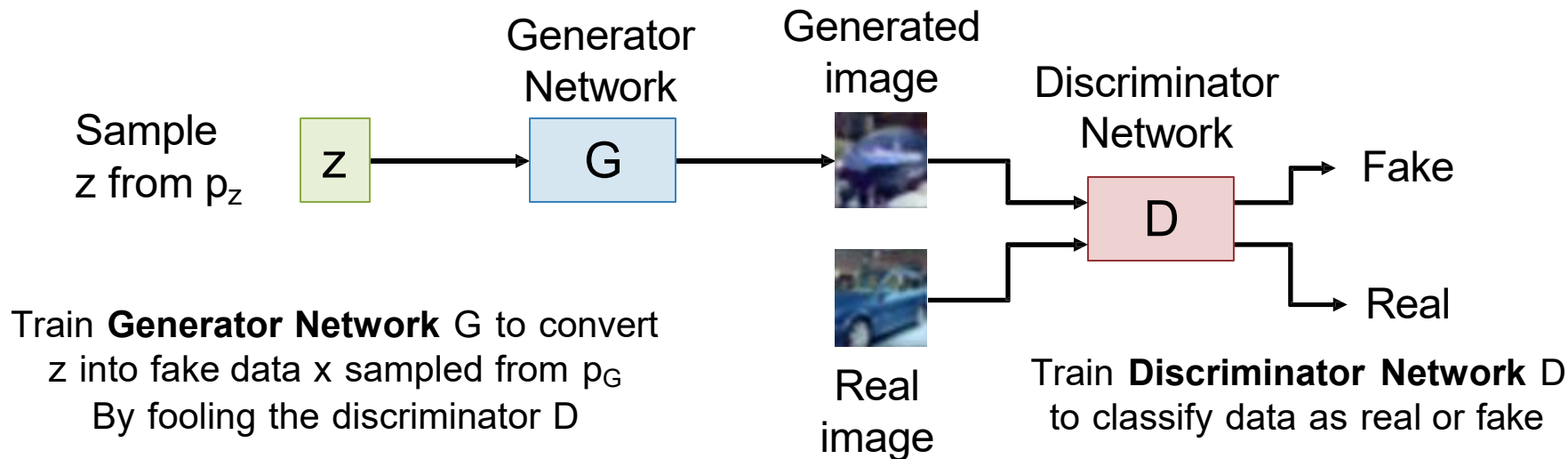
Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{\text{data}}(x)$. Want to sample from p_{data}

Idea: Introduce a latent variable z with simple prior $p(z)$ (e.g. unit Gaussian)

Sample $z \sim p(z)$ and pass to a **Generator Network** $x = G(z)$

Then x is a sample from the **Generator distribution** p_G . Want $p_G = p_{\text{data}}$!



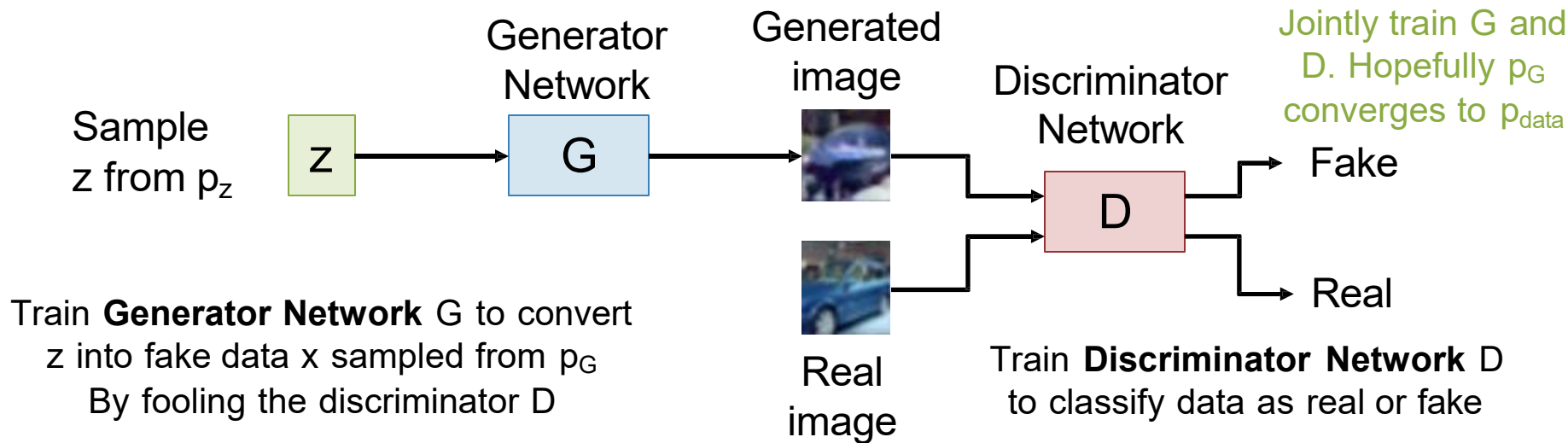
Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{\text{data}}(x)$. Want to sample from p_{data}

Idea: Introduce a latent variable z with simple prior $p(z)$ (e.g. unit Gaussian)

Sample $z \sim p(z)$ and pass to a **Generator Network** $x = G(z)$

Then x is a sample from the **Generator distribution** p_G . Want $p_G = p_{\text{data}}$!



Generative Adversarial Networks: Training Objective

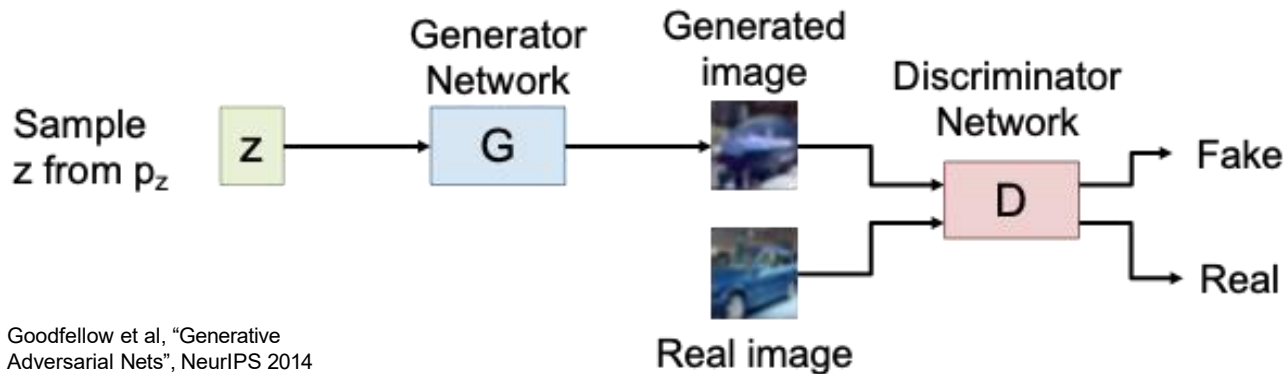
Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right)$$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$



$D(x) = P(x \text{ is real})$
 $D(x) = 0 \Rightarrow \text{fake}$
 $D(x) = 1 \Rightarrow \text{real}$

Generative Adversarial Networks: Training Objective

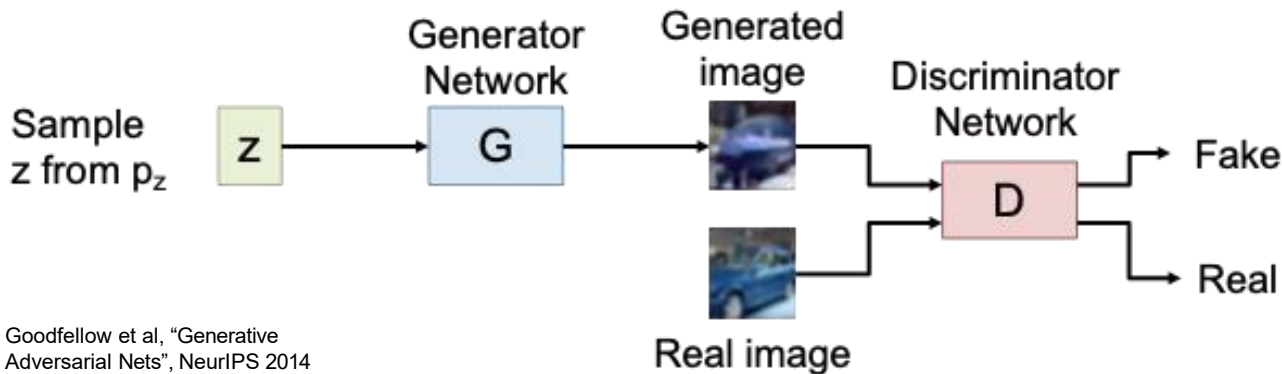
Jointly train generator G and discriminator D with a **minimax game**

Discriminator wants
 $D(x) = 1$ for real data

Discriminator wants
 $D(x) = 0$ for fake data

Imagine
fixing **G**

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log (1 - D(G(z))) \right] \right)$$



$D(x) = P(x \text{ is real})$
 $D(x) = 0 \Rightarrow \text{fake}$
 $D(x) = 1 \Rightarrow \text{real}$

Generative Adversarial Networks: Training Objective

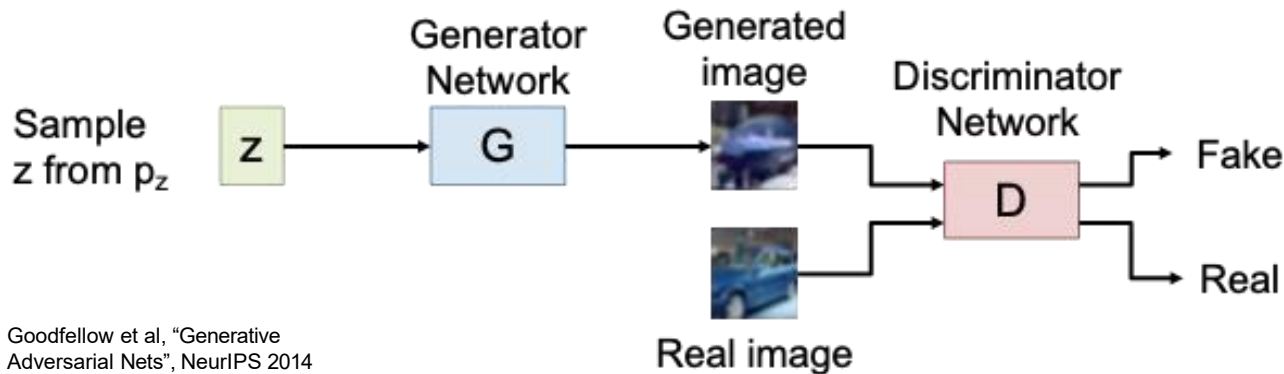
Jointly train generator G and discriminator D with a **minimax game**

Imagine fixing **D**

This term does not depend on G

Generator wants $D(x) = 1$ for fake data

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$



$D(x) = P(x \text{ is real})$
 $D(x) = 0 \Rightarrow \text{fake}$
 $D(x) = 1 \Rightarrow \text{real}$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$
$$= \min_{\mathbf{G}} \max_{\mathbf{D}} V(\mathbf{G}, \mathbf{D})$$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

$$= \min_G \max_D V(G, D)$$

While True:

$$D = D + \alpha_D \frac{dV}{dD}$$

$$G = G - \alpha_G \frac{dV}{dG}$$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

$$= \min_G \max_D V(G, D)$$

We are not minimizing any overall loss! No training curves to look at!

While True:

$$D = D + \alpha_D \frac{dV}{dD}$$

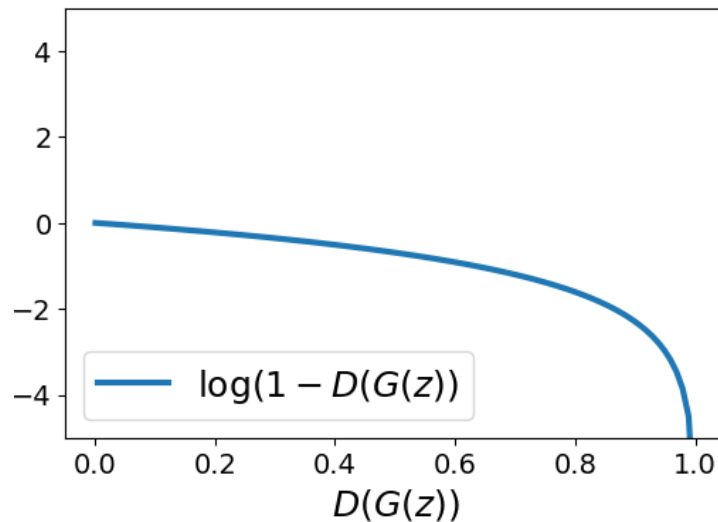
$$G = G - \alpha_G \frac{dV}{dG}$$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so $D(G(z))$ close to 0



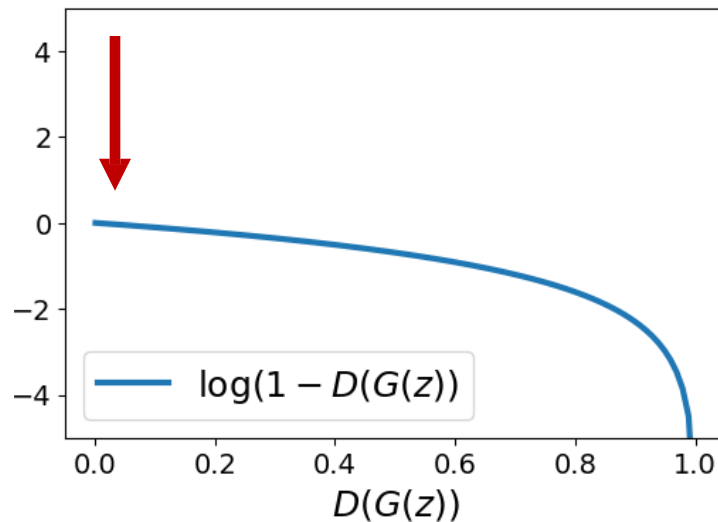
Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so $D(G(z))$ close to 0

Problem: Gradients for G are close to 0



Generative Adversarial Networks: Training Objective

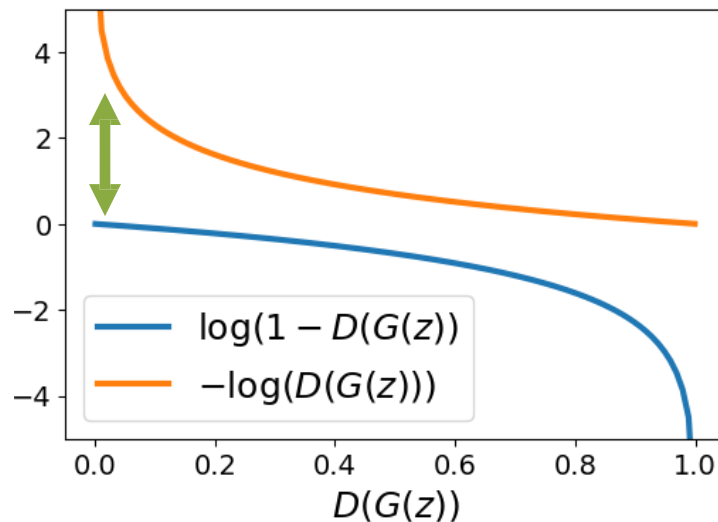
Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so $D(G(z))$ close to 0

Problem: Gradients for G are close to 0

Solution: Generator wants $D(G(z)) = 1$.
Train generator to minimize $-\log(D(G(z)))$
and discriminator to maximize $\log(1-D(G(z)))$
so generator gets strong gradients at start



Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

Why is this a good objective?

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

Why is this a good objective?

Inner objective is maximized by

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

(for any p_G)

(Proof omitted)

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

Why is this a good objective?

Inner objective is maximized by

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

(for any p_G)

Outer objective is then minimized by

$$p_G(x) = p_{data}(x)$$

(Proof omitted)

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

Why is this a good objective?

Inner objective is maximized by

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

(for any p_G)

(Proof omitted)

Outer objective is

then minimized by

$$p_G(x) = p_{data}(x)$$

Caveats:

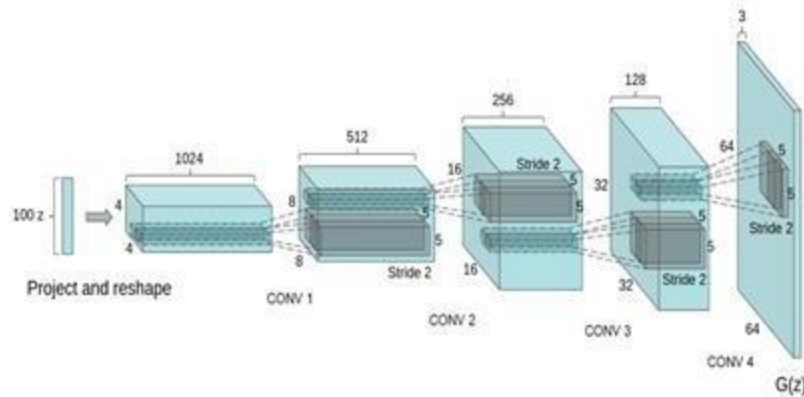
1. Neural nets with fixed capacity may not be able to represent optimal D and G
2. This tells us nothing about convergence to the solution with finite data

GAN Architectures: DC-GAN

Generator G and discriminator D are both neural networks

Usually CNNs ... GANs fell out of favor before ViT became popular

DC-GAN was the first GAN architecture that worked on non-toy data



GAN Architectures: DC-GAN

GPT-1 Paper (2018)

Improving Language Understanding
by Generative Pre-Training

Alec Radford
OpenAI
alec@openai.com

Karthik Narasimhan
OpenAI
karthikn@openai.com

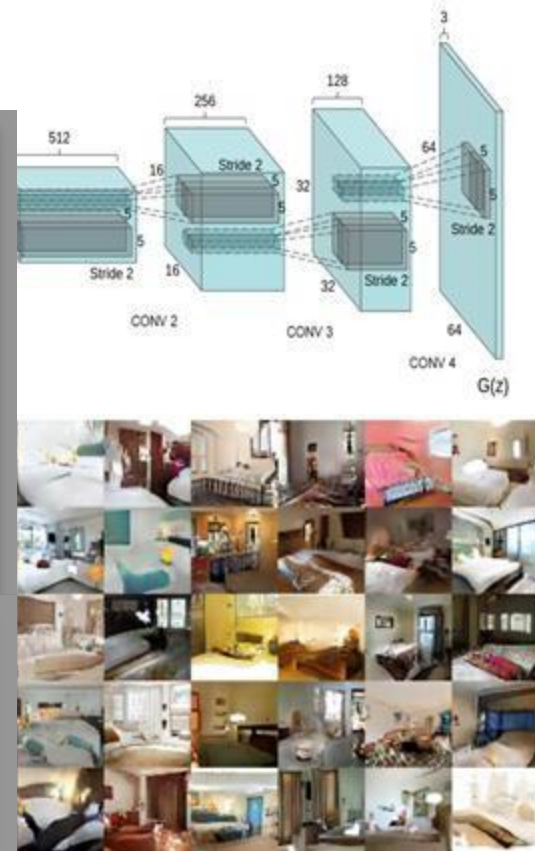
Tim Salimans
OpenAI
tim@openai.com

Ilya Sutskever
OpenAI
ilyasu@openai.com

GPT-2 Paper (2019)

Language Models are Unsupervised Multitask Learners

Alec Radford *¹ Jeffrey Wu *¹ Rewon Child¹ David Luan¹ Dario Amodei **¹ Ilya Sutskever **¹



Radford et al, ICLR 2016

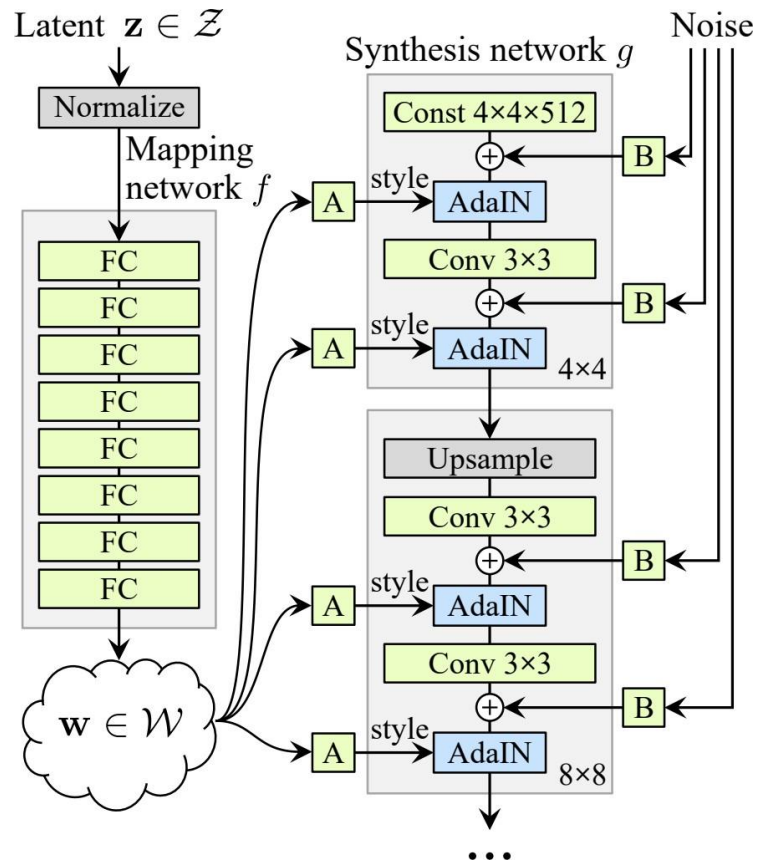
GAN Architectures: StyleGAN

Generator G and discriminator D are both neural networks

StyleGAN uses a more complex architecture that injects noise via **adaptive normalization**.

At each layer predict a scale w and shift b the same shape as x :

$$AdaIN(x, w, b)_i = w_i \frac{x_i - \mu(x)}{\sigma(x)} + b_i$$



Karras et al, "A Style-Based Generator Architecture for Generative Adversarial Networks", CVPR 2019

GANs: Latent Space Interpolation

Latent space is **smooth**.

Given latent vectors z_0 and z_1 , we can **interpolate** between them:

$$\begin{aligned}z_t &= tz_0 + (1 - t)z_1 \\x_t &= G(z_t)\end{aligned}$$

The resulting image x_t smoothly interpolate between samples!

GANs: Latent Space Interpolation

Latent space is **smooth**.

Given latent vectors z_0 and z_1 , we can **interpolate** between them:

$$z_t = tz_0 + (1 - t)z_1$$
$$x_t = G(z_t)$$

The resulting image x_t smoothly interpolate between samples!



Generative Adversarial Networks: Summary

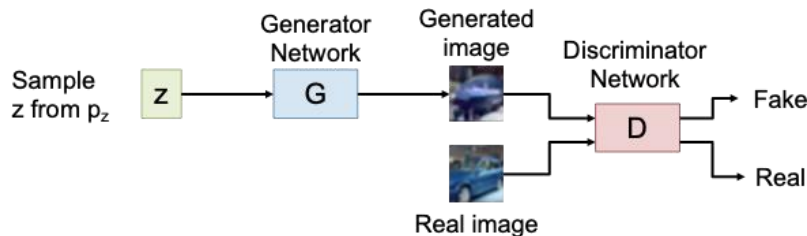
Jointly train Generator and Discriminator with a minimax game

Pros:

- Simple formulation
- Very good image quality

Cons:

- No loss curve to look at
- Unstable training
- Hard to scale to big models + data



These were the go-to generative models from ~2016 – 2021